

Report
v. 1.0

Customer
Shift DeFi



Smart Contract Audit

Shift DeFi Platform

29th December 2025



Report prepared by
ABDK
Consulting

Contents

1	Changelog	12
2	Summary	13
3	System overview	14
4	Methodology	17
5	Our findings	18
6	Major Issues	19
	CVF-1. FIXED	19
	CVF-2. FIXED	19
	CVF-3. FIXED	20
	CVF-4. INFO	20
	CVF-5. FIXED	20
	CVF-6. FIXED	20
	CVF-7. FIXED	21
	CVF-8. FIXED	21
	CVF-9. FIXED	21
	CVF-10. FIXED	22
	CVF-11. FIXED	22
	CVF-12. FIXED	22
	CVF-13. FIXED	23
	CVF-14. FIXED	23
	CVF-15. FIXED	23
	CVF-16. FIXED	24
7	Moderate Issues	25
	CVF-17. FIXED	25
	CVF-18. FIXED	25
	CVF-19. FIXED	25
	CVF-20. FIXED	26
	CVF-21. FIXED	26
	CVF-22. FIXED	26
	CVF-23. FIXED	26
	CVF-24. FIXED	27
	CVF-25. FIXED	27
	CVF-26. FIXED	27
	CVF-27. FIXED	28
	CVF-28. FIXED	28
	CVF-29. FIXED	28
	CVF-30. FIXED	29
	CVF-31. FIXED	29

CVF-32.	FIXED	29
CVF-33.	FIXED	30
CVF-34.	INFO	30
CVF-35.	FIXED	31
CVF-36.	FIXED	31
CVF-37.	INFO	31
CVF-38.	FIXED	32
CVF-39.	FIXED	32
CVF-40.	FIXED	32
CVF-41.	FIXED	32
CVF-42.	FIXED	33
CVF-43.	FIXED	33
CVF-44.	FIXED	33
CVF-45.	FIXED	34
CVF-46.	FIXED	34
CVF-47.	INFO	34
CVF-48.	FIXED	35
CVF-49.	INFO	35
CVF-50.	FIXED	35
CVF-51.	FIXED	36
CVF-52.	FIXED	36
CVF-53.	FIXED	36
CVF-54.	FIXED	36
CVF-55.	INFO	37
CVF-56.	FIXED	37
CVF-57.	FIXED	37
CVF-58.	INFO	38
CVF-59.	FIXED	38
CVF-60.	FIXED	38
CVF-61.	FIXED	38
CVF-62.	INFO	39
CVF-63.	FIXED	39
CVF-64.	INFO	39
CVF-65.	FIXED	40
CVF-66.	FIXED	40
CVF-67.	FIXED	41
CVF-68.	FIXED	41
CVF-69.	INFO	41
CVF-70.	FIXED	42
CVF-71.	FIXED	42
CVF-72.	INFO	43
CVF-73.	FIXED	43
CVF-74.	FIXED	43
CVF-75.	FIXED	44

8 Recommendations	45
CVF-76. INFO	45
CVF-77. INFO	45
CVF-78. INFO	45
CVF-79. INFO	46
CVF-80. INFO	46
CVF-81. INFO	46
CVF-82. INFO	46
CVF-83. INFO	46
CVF-84. INFO	47
CVF-85. FIXED	47
CVF-86. FIXED	47
CVF-87. INFO	47
CVF-88. INFO	48
CVF-89. INFO	48
CVF-90. INFO	48
CVF-91. INFO	48
CVF-92. INFO	49
CVF-93. INFO	49
CVF-94. INFO	49
CVF-95. FIXED	49
CVF-96. FIXED	50
CVF-97. FIXED	50
CVF-98. FIXED	50
CVF-99. FIXED	50
CVF-100. FIXED	50
CVF-101. FIXED	51
CVF-102. FIXED	51
CVF-103. FIXED	51
CVF-104. FIXED	51
CVF-105. FIXED	52
CVF-106. FIXED	52
CVF-107. FIXED	52
CVF-108. FIXED	52
CVF-109. FIXED	53
CVF-110. FIXED	53
CVF-111. FIXED	53
CVF-112. FIXED	53
CVF-113. FIXED	54
CVF-114. FIXED	54
CVF-115. INFO	54
CVF-116. INFO	55
CVF-117. INFO	55
CVF-118. INFO	55
CVF-119. INFO	55
CVF-120. FIXED	56

CVF-121. FIXED	56
CVF-122. FIXED	56
CVF-123. FIXED	56
CVF-124. FIXED	57
CVF-125. FIXED	57
CVF-126. FIXED	57
CVF-127. FIXED	57
CVF-128. FIXED	58
CVF-129. INFO	58
CVF-130. INFO	58
CVF-131. INFO	59
CVF-132. INFO	59
CVF-133. INFO	59
CVF-134. INFO	59
CVF-135. INFO	60
CVF-136. INFO	60
CVF-137. INFO	60
CVF-138. INFO	60
CVF-139. INFO	60
CVF-140. INFO	61
CVF-141. INFO	61
CVF-142. INFO	61
CVF-143. INFO	61
CVF-144. INFO	62
CVF-145. INFO	62
CVF-146. INFO	62
CVF-147. INFO	62
CVF-148. INFO	63
CVF-149. INFO	63
CVF-150. INFO	63
CVF-151. INFO	63
CVF-152. INFO	64
CVF-153. INFO	64
CVF-154. INFO	64
CVF-155. INFO	64
CVF-156. INFO	65
CVF-157. INFO	65
CVF-158. INFO	66
CVF-159. INFO	66
CVF-160. INFO	66
CVF-161. INFO	66
CVF-162. INFO	67
CVF-163. INFO	67
CVF-164. INFO	67
CVF-165. INFO	67
CVF-166. INFO	68

CVF-167. INFO	68
CVF-168. INFO	68
CVF-169. INFO	68
CVF-170. INFO	69
CVF-171. INFO	69
CVF-172. INFO	69
CVF-173. INFO	69
CVF-174. INFO	70
CVF-175. FIXED	70
CVF-176. INFO	70
CVF-177. INFO	70
CVF-178. INFO	71
CVF-179. INFO	71
CVF-180. INFO	71
CVF-181. INFO	72
CVF-182. INFO	72
CVF-183. INFO	73
CVF-184. INFO	73
CVF-185. INFO	73
CVF-186. INFO	74
CVF-187. INFO	74
CVF-188. INFO	74
CVF-189. INFO	74
CVF-190. INFO	75
CVF-191. INFO	75
CVF-192. INFO	75
CVF-193. INFO	75
CVF-194. INFO	76
CVF-195. INFO	76
CVF-196. INFO	76
CVF-197. INFO	77
CVF-198. INFO	77
CVF-199. INFO	77
CVF-200. INFO	77
CVF-201. INFO	78
CVF-202. INFO	78
CVF-203. INFO	78
CVF-204. INFO	79
CVF-205. INFO	79
CVF-206. INFO	79
CVF-207. INFO	79
CVF-208. INFO	80
CVF-209. INFO	80
CVF-210. INFO	80
CVF-211. INFO	80
CVF-212. INFO	81

CVF-213. INFO	81
CVF-214. INFO	81
CVF-215. INFO	81
CVF-216. INFO	82
CVF-217. INFO	82
CVF-218. INFO	82
CVF-219. INFO	83
CVF-220. INFO	84
CVF-221. INFO	84
CVF-222. INFO	84
CVF-223. INFO	84
CVF-224. INFO	85
CVF-225. INFO	85
CVF-226. INFO	85
CVF-227. INFO	86
CVF-228. INFO	86
CVF-229. INFO	86
CVF-230. INFO	87
CVF-231. INFO	87
CVF-232. INFO	87
CVF-233. INFO	88
CVF-234. INFO	88
CVF-235. INFO	88
CVF-236. INFO	88
CVF-237. INFO	89
CVF-238. INFO	89
CVF-239. INFO	89
CVF-240. INFO	89
CVF-241. FIXED	90
CVF-242. FIXED	90
CVF-243. INFO	90
CVF-244. INFO	91
CVF-245. INFO	91
CVF-246. INFO	91
CVF-247. FIXED	91
CVF-248. INFO	92
CVF-249. INFO	92
CVF-250. INFO	92
CVF-251. INFO	92
CVF-252. INFO	93
CVF-253. INFO	93
CVF-254. INFO	93
CVF-255. INFO	94
CVF-256. INFO	94
CVF-257. FIXED	94
CVF-258. FIXED	95

CVF-259. FIXED	95
CVF-260. FIXED	95
CVF-261. FIXED	95
CVF-262. FIXED	95
CVF-263. FIXED	96
CVF-264. FIXED	96
CVF-265. FIXED	96
CVF-266. FIXED	96
CVF-267. FIXED	97
CVF-268. FIXED	97
CVF-269. FIXED	97
CVF-270. FIXED	97
CVF-271. FIXED	98
CVF-272. FIXED	98
CVF-273. INFO	98
CVF-274. INFO	98
CVF-275. INFO	99
CVF-276. INFO	99
CVF-277. INFO	99
CVF-278. INFO	99
CVF-279. INFO	99
CVF-280. INFO	100
CVF-281. INFO	100
CVF-282. INFO	100
CVF-283. INFO	100
CVF-284. INFO	100
CVF-285. INFO	101
CVF-286. INFO	101
CVF-287. INFO	101
CVF-288. INFO	101
CVF-289. INFO	102
CVF-290. INFO	102
CVF-291. INFO	102
CVF-292. INFO	103
CVF-293. INFO	103
CVF-294. INFO	103
CVF-295. INFO	104
CVF-296. INFO	104
CVF-297. INFO	104
CVF-298. INFO	105
CVF-299. INFO	105
CVF-300. INFO	106
CVF-301. INFO	106
CVF-302. INFO	107
CVF-303. INFO	107
CVF-304. INFO	107

CVF-305. INFO	107
CVF-306. INFO	107
CVF-307. INFO	108
CVF-308. INFO	108
CVF-309. INFO	108
CVF-310. INFO	108
CVF-311. INFO	109
CVF-312. INFO	109
CVF-313. INFO	109
CVF-314. INFO	109
CVF-315. INFO	110
CVF-316. INFO	110
CVF-317. INFO	111
CVF-318. INFO	111
CVF-319. INFO	111
CVF-320. INFO	111
CVF-321. INFO	112
CVF-322. INFO	112
CVF-323. INFO	112
CVF-324. INFO	112
CVF-325. INFO	113
CVF-326. INFO	113
CVF-327. INFO	113
CVF-328. INFO	113
CVF-329. INFO	114
CVF-330. INFO	114
CVF-331. INFO	114
CVF-332. INFO	114
CVF-333. INFO	115
CVF-334. INFO	115
CVF-335. INFO	115
CVF-336. INFO	115
CVF-337. INFO	116
CVF-338. INFO	116
CVF-339. INFO	116
CVF-340. INFO	116
CVF-341. INFO	117
CVF-342. INFO	117
CVF-343. INFO	117
CVF-344. INFO	117
CVF-345. INFO	118
CVF-346. INFO	118
CVF-347. INFO	118
CVF-348. INFO	118
CVF-349. INFO	119
CVF-350. INFO	119

CVF-351. INFO	119
CVF-352. INFO	120
CVF-353. INFO	120
CVF-354. INFO	120
CVF-355. INFO	120
CVF-356. INFO	121
CVF-357. INFO	121
CVF-358. INFO	121
CVF-359. INFO	121
CVF-360. INFO	121
CVF-361. INFO	122
CVF-362. INFO	122
CVF-363. INFO	122
CVF-364. INFO	122
CVF-365. INFO	123
CVF-366. INFO	123
CVF-367. INFO	123
CVF-368. INFO	123
CVF-369. INFO	123
CVF-370. INFO	124
CVF-371. INFO	124
CVF-372. INFO	124
CVF-373. INFO	124
CVF-374. INFO	125
CVF-375. INFO	125
CVF-376. INFO	125
CVF-377. INFO	126
CVF-378. INFO	126
CVF-379. INFO	126
CVF-380. INFO	126
CVF-381. INFO	127
CVF-382. INFO	127
CVF-383. INFO	127
CVF-384. INFO	127
CVF-385. INFO	128
CVF-386. FIXED	128
CVF-387. INFO	128
CVF-388. FIXED	128
CVF-389. INFO	129
CVF-390. INFO	129
CVF-391. INFO	129
CVF-392. INFO	130
CVF-393. INFO	130
CVF-394. INFO	130
CVF-395. INFO	131
CVF-396. INFO	132

CVF-397. INFO	132
CVF-398. INFO	132
CVF-399. INFO	133
CVF-400. INFO	133
CVF-401. INFO	133
CVF-402. INFO	133
CVF-403. INFO	134
CVF-404. INFO	134
CVF-405. INFO	134

1 Changelog

#	Date	Author	Description
0.1	29.12.25	A. Zveryanskaya	Initial Draft
0.2	29.12.25	A. Zveryanskaya	Minor revision
1.0	29.12.25	A. Zveryanskaya	Release

2 Summary

All modifications to this document are prohibited. Violators will be prosecuted to the full extent of the U.S. law.

This document presents the results of a smart contract audit performed by ABDK Consulting for Shift DeFi platform. The audit was conducted by Mikhail Vladimirov and Dmitry Khovratovich between 16th October and 29th December 2025. The goal of the audit was to identify security vulnerabilities, logic flaws, and code quality issues across the System's architecture, including vault management, cross-chain operations, strategy execution, token swapping, and bridge integrations.

Our conclusion is that the System demonstrates a mature architecture with well-structured contract separation and thoughtful design patterns. The codebase leverages OpenZeppelin's upgradeable contracts and integrates with established DeFi protocols including [UniswapV3](#), [linch](#), [Compound V3](#), [Across V3](#), and [Curve](#). The System implements a sophisticated multi-layer architecture with vault shares, container orchestration, strategy templates, and cross-chain messaging capabilities. Code quality is generally solid with proper use of access control, reentrancy guards, and safe token operations.

The audit identified 16 major issues. The most critical findings included unreliable balance checking in swap adapters [CVF-1](#), [CVF-2](#), and [CVF-14](#), which could allow manipulation through token transfer callbacks. A significant vulnerability in [MessageRouter CVF-13](#) allowed nonce resets that could enable replay attacks. Additional major issues included incorrect NAV delta validation logic [CVF-8](#) and [CVF-12](#), strategy bitmask limitations [CVF-9](#), whitelist check flaws preventing legitimate operations [CVF-10](#), and improper rounding in share calculations [CVF-15](#). Moderate issues primarily concerned inefficient data handling, suboptimal validation checks, and procedural improvements.

We recommend prioritizing the correction of all major issues, particularly those affecting cross-chain security and token accounting accuracy. The team should implement proper return value handling for swap operations, eliminate nonce reset capabilities, correct validation logic throughout strategy operations, and optimize data copying operations for gas efficiency. Overall, with the identified issues addressed, the System demonstrates strong potential for secure production deployment.

It is important to note that a security audit is not a guarantee of absolute security but rather a snapshot of the system's security posture at a specific point in time. While we strive to uncover all potential issues, the evolving nature of blockchain technology and DeFi means new vulnerabilities can emerge.

3 System overview

This section provides a high-level overview of the Shift DeFi platform, a cross-chain yield aggregation protocol designed to enable users to deposit assets into a vault and earn returns through automated deployment across multiple blockchain networks and DeFi strategies. The System manages user deposits through a share-based tokenized vault architecture, coordinates asset allocation across different chains, executes yield-generating strategies, and facilitates cross-chain communication and bridging to maximize capital efficiency.

- [Original Code](#)
- [Code with Fixes](#)

Files:

/		
Vault.sol	SwapRouter.sol	StrategyTemplate.sol
StrategyContainer.sol	ReshufflingGateway.sol	PriceOracle.sol
MessageRouter.sol	CrossChainContainer.sol	ContainerPrincipal.sol
ContainerLocal.sol	ContainerAgent.sol	Container.sol
Collector.sol	BridgeAdapter.sol	
swaps/		
UniversalAdapter.sol	UniswapV3Adapter.sol	OneInchAdapter.sol
strategies/		
CompoundV3.sol		
libraries/		
Withdrawal ResponseLib.sol	Withdrawal RequestLib.sol	MessageCodec.sol
DepositResponseLib.sol	DepositRequestLib.sol	ContainerMessage PackingLib.sol
AddressArray PackingLib.sol		

libraries/helpers/		
RingCacheLibrary.sol	Errors.sol	EnumerableAddress SetExtended.sol
Common.sol		
interfaces/		
IVault.sol	IUniversalAdapter.sol	IUniswapV3Adapter.sol
ISwapRouter.sol	ISwapAdapter.sol	IStrategyTemplate.sol
IStrategyContainer.sol		

The System is implemented in Solidity version 0.8.28, utilizing the latest language features and compiler optimizations available at the time of development. The codebase employs OpenZeppelin's battle-tested upgradeable contract libraries extensively, including upgradeable versions of access control mechanisms, ERC20 token standards, reentrancy protection, and proxy initialization patterns. This choice enables the System to support future protocol upgrades while maintaining state continuity. The development approach follows modern Solidity best practices, including comprehensive use of SafeERC20 for token transfers, the Math library for precision arithmetic operations, and EnumerableSet for efficient collection management.

At the architectural core lies **Vault**, which serves as the primary user interaction point. Users deposit a designated notion token into the vault and receive proportional shares represented as ERC20 tokens. The vault implements a batch processing mechanism for both deposits and withdrawals, aggregating multiple user requests to optimize gas costs and enable efficient cross-chain coordination. The vault maintains connections to multiple containers, each assigned specific weight allocations that determine capital distribution. Container weights are configurable by authorized roles, allowing dynamic rebalancing of capital across different deployment targets. The vault tracks net asset values at multiple stages of the deposit and withdrawal lifecycle, ensuring accurate share price calculations that reflect the underlying value of deployed assets.

The container layer provides three distinct implementations addressing different deployment scenarios. **ContainerLocal** handles strategies deployed on the same blockchain as the vault, providing a straightforward execution path without cross-chain complexity. **ContainerPrincipal** and **ContainerAgent** work together to enable cross-chain deployments, where the principal resides on the vault's chain and coordinates with agent containers on remote chains. These cross-chain containers implement a message-passing protocol for deposit and withdrawal requests, ensuring synchronized state transitions across networks. Each container manages whitelisted tokens, implements role-based access control for operational activities, and maintains connections to swap routing infrastructure for token conversions.

Strategy execution is abstracted through **StrategyTemplate**, which defines a common interface for entering and exiting positions, calculating allocated net asset values, and handling emergency withdrawals. The System includes a concrete implementation for Compound V3 lending markets, demonstrating integration with external yield protocols. Strategies define input and output tokens, track state through identifiers, and implement state-specific exit logic. The template pattern enables strategy containers to manage

multiple strategies simultaneously, coordinating entry and exit operations through bitmask tracking mechanisms. Strategy containers calculate aggregate net asset values across all deployed strategies, collect performance fees for the treasury, and support emergency exit capabilities for risk mitigation.

Cross-chain communication infrastructure centers on **MessageRouter**, which manages message delivery between containers across different blockchains. The router implements path-based whitelisting, connecting sender addresses and destination chains to specific receiver contracts. Nonce tracking prevents replay attacks by ensuring messages are processed in order. The router supports multiple message adapter integrations, allowing the System to leverage different cross-chain communication protocols. Bridge operations are abstracted through **BridgeAdapter**, with a concrete implementation for Across V3 protocol. Bridge adapters maintain path mappings between tokens on source and destination chains, validate bridge instructions against configured parameters, and implement claimable amount tracking for asynchronous bridge completion.

Token swapping capabilities are provided through **SwapRouter**, which routes swap requests to whitelisted adapter contracts. The System implements adapters for Uniswap V3 and 1inch aggregator, enabling strategies to convert rewards and rebalance token positions. The swap router validates minimum output amounts, transfers tokens securely, and emits events for off-chain monitoring. Price oracle functionality relies on Chainlink data feeds through **PriceOracle**, which provides USD valuations for net asset value calculations. The oracle maintains mappings from token addresses to Chainlink feed contracts, retrieves latest prices with decimal precision, and converts token amounts to unified USD denominations for consistent accounting across different tokens.

Additional infrastructure includes **ReshufflingGateway** for managed container reconfigurations, supporting both reshuffling operations that migrate capital between containers and repair operations for emergency recovery scenarios. The gateway coordinates with the vault to claim assets, perform necessary token swaps, and redistribute capital according to updated allocations. **Collector** provides a simple claiming interface for users to retrieve accepted notion tokens from processed batches. Custom libraries throughout the codebase handle message encoding and decoding, address array packing for efficient cross-chain transmission, ring cache implementations for deduplication, and helper utilities for decimal normalization and error definitions.

4 Methodology

The methodology is not a strict formal procedure, but rather a selection of methods and tactics combined differently and tuned for each particular project, depending on the project structure and technologies used, as well as on client expectations from the audit.

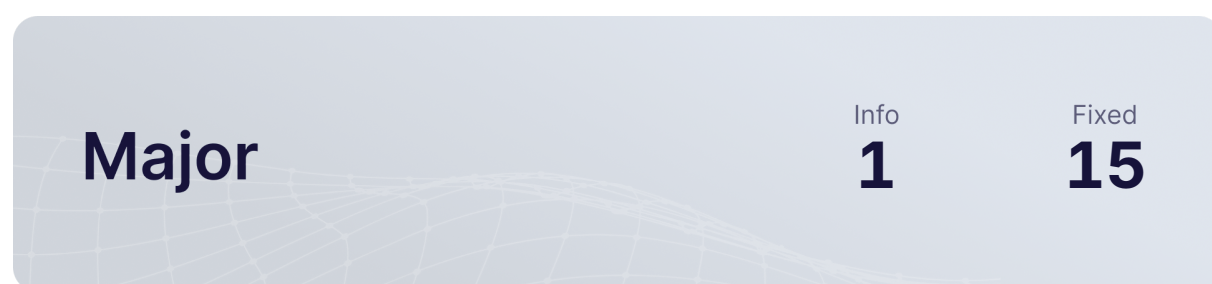
- **General Code Assessment.** The code is reviewed for clarity, consistency, style, and for whether it follows best code practices applicable to the particular programming language used. We check indentation, naming convention, commented code blocks, code duplication, confusing names, confusing, irrelevant, or missing comments etc. At this phase we also understand overall code structure.
- **Entity Usage Analysis.** Usages of various entities defined in the code are analysed. This includes both: internal usages from other parts of the code as well as potential external usages. We check that entities are defined in proper places as well as their visibility scopes and access levels are relevant. At this phase, we understand overall system architecture and how different parts of the code are related to each other.
- **Access Control Analysis.** For those entities, that could be accessed externally, access control measures are analysed. We check that access control is relevant and done properly. At this phase, we understand user roles and permissions, as well as what assets the system ought to protect.
- **Code Logic Analysis.** The code logic of particular functions is analysed for correctness and efficiency. We check whether the code actually does what it is supposed to do, whether the algorithms are optimal and correct, and whether proper data types are used. We also make sure that external libraries used in the code are up to date and relevant to the tasks they solve in the code. At this phase we also understand data structures used and the purposes they are used for.

We classify issues by the following severity levels:

- **Critical issue** directly affects the smart contract functionality and may cause a significant loss.
- **Major issue** is either a solid performance problem or a sign of misuse: a slight code modification or environment change may lead to loss of funds or data. Sometimes it is an abuse of unclear code behaviour which should be double checked.
- **Moderate issue** is not an immediate problem, but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain business flows.
- **Recommendations** cover code style, best practices and general improvements.

5 Our findings

We found 16 major, and a few less important issues. All identified Major issues have been fixed.



Fixed 15 out of 16 issues

6 Major Issues

CVF-1 FIXED

- **Category** Flaw

- **Source** OneInchAdapter.sol

Description Checking the receiver's balance change is unreliable, as the receiver could have a transfer callback for the tokens, which could forward the received tokens elsewhere or modify the receiver's balance in some other way.

Recommendation Rely on the values returned by the called functions to tell the actual output amount.

```
32 uint256 balanceBefore = IERC20(tokenOut).balanceOf(receiver);
```

```
53 uint256 balanceAfter = IERC20(tokenOut).balanceOf(receiver);
    require(
        balanceAfter - balanceBefore >= minAmountOut,
        SlippageNotMet(tokenOut, balanceAfter - balanceBefore,
            ↪ minAmountOut)
    );
```

CVF-2 FIXED

- **Category** Flaw

- **Source** UniswapV3Adapter.sol

Description Checking the receiver's balance change is unreliable, as the receiver could have a transfer callback for the tokens, which could forward the received tokens elsewhere or modify the receiver's balance in some other way.

Recommendation Rely on the values returned by the called functions to tell the actual output amount.

```
54 uint256 balanceBefore = IERC20(tokenOut).balanceOf(receiver);
```

```
64 uint256 balanceAfter = IERC20(tokenOut).balanceOf(receiver);
    require(
        balanceAfter - balanceBefore >= minAmountOut,
        SlippageNotMet(tokenOut, balanceAfter - balanceBefore,
            ↪ minAmountOut)
    );
```

CVF-3 FIXED

- **Category** Procedural
- **Source** CompoundV3.sol

Recommendation This TODO should be resolved or removed. Testing logic should be removed from production code.

```
111 // TODO: just for tests, use adapter
```

CVF-4 INFO

- **Category** Unclear behavior
- **Source** RingCacheLibrary.sol

Description This logic skips zero values in cache.

Recommendation Properly return zero values from the cache.

```
63 if (_cache.ring[index] != bytes32(0)) {
```

CVF-5 FIXED

- **Category** Unclear behavior
- **Source** AcrossBridgeAdapter.sol

Description It is weird that slippage is checked against fee cap here.

Recommendation Either use a separate cap for slippage, or rename "feeCapPct" to something more generic.

```
77 require(  
    slippageDeltaPct < feeCapPct,  
    MinTokenAmountTooLow(instruction.amount, acrossParams.fee,  
        ↪ instruction.minTokenAmount)  
80 );
```

CVF-6 FIXED

- **Category** Unclear behavior
- **Source** BridgeAdapter.sol

Recommendation It should be "<=" instead of ">=" here.

```
129 require(amount >= amountOnContract, Errors.NotEnoughTokens(token,  
    ↪ amount));
```

CVF-7 FIXED

- **Category** Flaw

- **Source** BridgeAdapter.sol

Description This will revert in case “amount” is greater than the current contract’s balance.

Recommendation Allow claiming less tokens in case the full amount cannot be claimed.

Client Comment *This behavior is intentional by design. The contract assumes the invariant that claimableAmounts[claimer][token] is always fully backed by the contract’s token balance. If, for any reason, amount exceeds the contract’s balance, this indicates a misconfiguration, an upstream accounting error, or an abnormal bridge condition that must be handled via governance / emergency procedures rather than by silently allowing partial withdrawals and breaking further platform accounting.*

```
133 IERC20(token).safeTransfer(claimer, amount);
```

CVF-8 FIXED

- **Category** Suboptimal

- **Source** StrategyTemplate.sol

Description Checking for the minimum NAV delta on exit doesn’t make much sense, as the less NAV delta is the better.

Recommendation Check for the maximum NAV delta instead.

```
161 require(  
    vars.allocatedNavBeforeExit - vars.allocatedNavAfterExit >  
        ↪ minNavDelta,  
    SlippageCheckFailed(vars.allocatedNavBeforeExit, vars.  
        ↪ allocatedNavAfterExit)  
);
```

CVF-9 FIXED

- **Category** Unclear behavior

- **Source** StrategyContainer.sol

Description This won’t work in case there are more than 255 strategies.

Recommendation Either properly support arbitrary number of strategies, or explicitly forbid more than 255 strategies.

```
184 return _strategyEnterBitmask == (1 << _strategies.length()) - 1;
```

```
273 return _strategyExitBitmask == (1 << _strategies.length()) - 1;
```

CVF-10 FIXED

- **Category** Flaw

- **Source** StrategyContainer.sol

Description This makes it impossible to take even whitelisted tokens from a strategy if some other tokens in the same strategy are not whitelisted.

Recommendation Don't check against the white list on exit.

```
288 for (uint256 i = 0; i < length; ++i) {  
    require(!_isTokenWhitelisted(tokens[i]), NotWhitelistedToken(  
        ↪ tokens[i]));  
290 IERC20(tokens[i]).safeTransferFrom(strategy, address(this),  
    ↪ amounts[i]);
```

```
312 for (uint256 i = 0; i < length; ++i) {  
    require(!_isTokenWhitelisted(tokens[i]), NotWhitelistedToken(  
        ↪ tokens[i]));  
    IERC20(tokens[i]).safeTransferFrom(strategy, address(this),  
        ↪ amounts[i]);
```

CVF-11 FIXED

- **Category** Suboptimal

- **Source** CrossChainContainer.sol

Description This should be done only when the new expected amount for the token is non zero.

```
101 claimCounterCached += 1;
```

CVF-12 FIXED

- **Category** Suboptimal

- **Source** ContainerLocal.sol

Description Specifying the minimum NAV delta when exiting a strategy doesn't make much sense, as the less is the delta the better.

Recommendation Specify the maximum NAV delta instead.

```
111 function exitStrategy(address strategy, uint256 minNavDelta)  
    ↪ external onlyRole(OPERATOR_ROLE) {
```

```
125     uint256[] calldata minNavDeltas
```

CVF-13 FIXED

- **Category** Flaw

- **Source** MessageRouter.sol

Description This logic allows resetting nonce for a path by blacklisting and then whitelisting it, and resetting nonce may allow replay attacks.

Recommendation Never reset nonce for a path.

```
57 _lastNonces[path] = 0;
```

```
72 _lastNonces[path] = 0;
```

CVF-14 FIXED

- **Category** Flaw

- **Source** SwapRouter.sol

Description Relying on the recipient's balance change is a bad practice, as the recipient may implement token transfer callback and forward received tokens synchronously.

Recommendation Use actual transferred and swapped amounts to estimate the amount of tokens got by the recipient.

```
52 uint256 amountOutBefore = IERC20(instruction.tokenOut).balanceOf(msg  
    ↪ .sender);
```

```
61 uint256 amountOutAfter = IERC20(instruction.tokenOut).balanceOf(msg.  
    ↪ sender);
```

```
68 return amountOutAfter - amountOutBefore; // cache
```

CVF-15 FIXED

- **Category** Flaw

- **Source** Vault.sol

Description Incrementing both, the numerator and the denominator by one is not a proper way to round the result up.

Recommendation Do proper rounding up instead.

```
513 vars.batchShares = vars.batchDeltaNav.mulDiv(totalSupply() + 1, vars  
    ↪ .totalNav0 + 1);
```

CVF-16 FIXED

- **Category** Flaw

- **Source** Vault.sol

Description This will revert in case there are 256 containers.

Recommendation Either reduce the maximum number of containers to 255, or put this line into an "unchecked" block.

Client Comment *Not an issue. The number of containers is already explicitly limited to 255 (at function addContainer: require(length <= MAX_CONTAINERS, MaxContainersReached()));. Therefore, a situation where _containers.length() == 256 is impossible by design, and this expression cannot overflow.*

```
527 return _withdrawReportBitmask == (1 << _containers.length()) - 1;
```



7 Moderate Issues

CVF-17 FIXED

- **Category** Bad datatype
- **Source** StrategyTemplate.sol

Recommendation The argument type should be "ISwapRouter".

```
197 function _claimAndSwapRewards(address swapRouter) internal virtual  
    ↪ {}
```

CVF-18 FIXED

- **Category** Bad naming
- **Source** ContainerAgent.sol

Description The check doesn't match the error.

Recommendation Fix the check, or throw another error, or remove the check.

```
120 require(remoteChainId > 0, Errors.ZeroAmount());
```

```
171 require(remoteChainId > 0, Errors.ZeroAmount());
```

CVF-19 FIXED

- **Category** Bad naming
- **Source** ContainerPrincipal.sol

Description The check doesn't match the error.

Recommendation Fix the check, or use a different error, or remove the check.

```
75 require(remoteChainId > 0, Errors.ZeroAmount());
```

```
113 require(remoteChainId > 0, Errors.ZeroAmount());
```

CVF-20 FIXED

- **Category** Procedural
- **Source** UniversalAdapter.sol

Recommendation A common good practice is to use a placeholder for the data that ought to be substituted, and then overwrite this placeholder with the real data. Such a approach doesn't require copying the data as it doesn't change data length.

```
70 bytes memory executionData = _insertData(swapInfo.payload, swapInfo.  
    ↪ amountIndex, abi.encode(amountIn));
```

CVF-21 FIXED

- **Category** Suboptimal
- **Source** UniversalAdapter.sol

Description This logic is inefficient as it copies data several tiems.

Recommendation Doo all operations in one pass: allocate the resulting array, then copy all parts into it.

```
80 return bytes.concat(bytes.concat(_data.slice(0, _index), _newData),  
    ↪ _data.slice(_index + 32, _data.length));
```

CVF-22 FIXED

- **Category** Unclear behavior
- **Source** CompoundV3.sol

Description This line should be executed only when "stateId" is "COMPOUND_V3".

```
25 uint256 allocatedUnderlying = IERC20(cometV3).balanceOf(address(this  
    ↪ ));
```

CVF-23 FIXED

- **Category** Suboptimal
- **Source** CompoundV3.sol

Description Here values just written into the storage are read back, which is inefficient.

Recommendation Reuse the written values.

```
39 cometV3Underlying = ICometV3(cometV3).baseToken();
```

```
43 _inputTokens[0] = cometV3Underlying;  
    _outputTokens[0] = cometV3Underlying;
```

CVF-24 FIXED

- **Category** Procedural

- **Source** CompoundV3.sol

Recommendation These two swaps should be merged into a single call to the Uniswap router.

```
121 IUniswapV3Router(UNISWAP_V3_ROUTER).exactInputSingle(  
    ↪ rewardToWethParams);
```

```
133 IUniswapV3Router(UNISWAP_V3_ROUTER).exactInputSingle(  
    ↪ wethToUnderlyingParams);
```

CVF-25 FIXED

- **Category** Suboptimal

- **Source** RingCacheLibrary.sol

Recommendation This logic is overcomplicated. It could be simplified like this:
uint256 index = (_cache.head++) % _cache.maxCacheSize; bytes32 oldValue = _cache.ring [index]; if (_cache.indices [oldValue] == index + 1) delete _cache.indices [oldValue]; else _cache.size++; _cache.indexes [value] = index + 1; _cache.ring [index] = value;

```
31 if (_cache.size < _cache.maxCacheSize) {  
    uint256 index = (_cache.head + _cache.size) % _cache.  
    ↪ maxCacheSize;
```

```
36 } else {
```

CVF-26 FIXED

- **Category** Unclear behavior

- **Source** RingCacheLibrary.sol

Description This condition should always be true.

Recommendation Remove the conditional operator or replace it with as assert.

```
49 if (_cache.ring[index] == value) {
```

CVF-27 FIXED

- **Category** Suboptimal

- **Source** Common.sol

Recommendation With denominator always equal to 1, there is no need to use the “muldiv” function here. Just use plain multiplication instead.

```
21 return amount.mulDiv(10 ** (UNIFIED_DECIMALS - decimals), ONE, Math.  
    ↪ Rounding.Floor);
```

CVF-28 FIXED

- **Category** Suboptimal

- **Source**
ContainerMessagePackingLib.sol

Description This copies the whole payload just to prepend a single byte to it. Such approach is inefficient.

Recommendation Refactor the code to prepend the byte in-place before sending the message. This would screw up the length field of the message, but the length could be read before prepending.

```
24 return abi.encodePacked(message.type_, message.payload);
```

CVF-29 FIXED

- **Category** Documentation

- **Source**
ContainerMessagePackingLib.sol

Description This corrupts the “data” array, which could be seen from the outside of the call.

Recommendation Clearly explain this behavior in a documentation comment.

```
34 mstore(payload, sub(mload(data), 1)) // Устанавливаем длину data
```

CVF-30 FIXED

- **Category** Procedural

- **Source** DepositRequestLib.sol

Description The request data is copied three times here: once inside the "packAddresses" call, the second time inside abi.encode, and another time in "ContainerMessagePackingLib.encode". This is inefficient.

Recommendation Refactor the code to avoid excessive copying.

```
20 bytes memory payload = abi.encode(request.tokens.packAddresses(),  
    ↪ request.amounts);  
  
24 .encode();
```

CVF-31 FIXED

- **Category** Procedural

- **Source** DepositRequestLib.sol

Description The request data is copied two times here: once inside "abi.decode" and another time inside "unpackAddresses".

Recommendation Refactor the code to avoid excessive copying.

```
37 (bytes memory packedAddresses, uint256[] memory amounts) = abi.  
    ↪ decode(payload, (bytes, uint256[]));  
    return DepositRequest({tokens: packedAddresses.unpackAddresses(),  
    ↪ amounts: amounts});
```

CVF-32 FIXED

- **Category** Procedural

- **Source** DepositResponseLib.sol

Description The request data is copied three times here: once inside the "packAddresses" call, the second time inside abi.encode, and another time in "ContainerMessagePackingLib.encode". This is inefficient.

Recommendation Refactor the code to avoid excessive copying.

```
22 bytes memory payload = abi.encode(  
    response.tokens.packAddresses(),  
  
29 ContainerMessagePackingLib  
30 .ContainerMessage({type_: ContainerMessagePackingLib.  
    ↪ DEPOSIT_RESPONSE_TYPE, payload: payload})  
    .encode();
```

CVF-33 FIXED

- **Category** Procedural

- **Source** DepositResponseLib.sol

Description The request data is copied two times here: once inside "abi.decode" and another time inside "unpackAddresses".

Recommendation Refactor the code to avoid excessive copying.

```
44 (bytes memory packedAddresses, uint256[] memory amounts, uint256
    ↪ navAH, uint256 navAE) = abi.decode(
49     DepositResponse({tokens: packedAddresses.unpackAddresses(),
    ↪ amounts: amounts, navAH: navAH, navAE: navAE});
```

CVF-34 INFO

- **Category** Procedural

- **Source** MessageCodec.sol

Description Copying the message data is inefficient.

Recommendation Consider returning message data in-place by overwriting the "pathOnRemoteChain" value (which was already read) with "messageLength" and returning the pointer the resulting "messageData" image in memory.

Client Comment *The suggested approach relies on overwriting an existing memory region that still contains live data (pathOnRemoteChain). This creates a high-risk side effect, breaks memory safety assumptions, and reduces code readability and maintainability. The current implementation, which allocates a new bytes array, guarantees data integrity and avoids unintended mutations of previously loaded values.*

```
32 bytes memory messageData = new bytes(messageLength);
```

CVF-35 FIXED

- **Category** Procedural

- **Source** WithdrawalResponseLib.sol

Description The request data is copied three times here: once inside the “packAddresses” call, the second time inside abi.encode, and another time in “ContainerMessagePackingLib.encode”. This is inefficient.

Recommendation Refactor the code to avoid excessive copying.

```
20 bytes memory payload = abi.encode(response.tokens.packAddresses(),  
    ↳ response.amounts);  
  
22 ContainerMessagePackingLib  
    .ContainerMessage({type_: ContainerMessagePackingLib.  
    ↳ WITHDRAWAL_RESPONSE_TYPE, payload: payload})  
    .encode();
```

CVF-36 FIXED

- **Category** Procedural

- **Source** WithdrawalResponseLib.sol

Description The request data is copied two times here: once inside “abi.decode” and another time inside “unpackAddresses”.

Recommendation Refactor the code to avoid excessive copying.

```
39 (bytes memory packedAddresses, uint256[] memory amounts) = abi.  
    ↳ decode(payload, (bytes, uint256[]));  
40 return WithdrawalResponse({tokens: packedAddresses.unpackAddresses()  
    ↳ , amounts: amounts});
```

CVF-37 INFO

- **Category** Suboptimal

- **Source** IPriceOracle.sol

Description Using variable number decimals is inefficient makes code overcomplicated.

Recommendation Consider using fixed number of decimals.

Client Comment Chainlink oracles are not limited to 8 decimals — different feeds use different precision values. Therefore, relying on a fixed decimals assumption would break compatibility with valid Chainlink oracle configurations. Keeping decimals variable ensures correct support for all feeds.

```
5 function getPrice(address token) external view returns (uint256  
    ↳ price, uint256 decimals);
```

CVF-38 FIXED

- **Category** Procedural

- **Source** IBridgeAdapter.sol

Recommendation The “chain” parameter should be indexed.

```
13 event BridgeSent(address indexed token, uint256 amount, uint256  
    ↪ chain);
```

CVF-39 FIXED

- **Category** Unclear behavior

- **Source** IBridgeAdapter.sol

Recommendation This function should return the amount claimed.

```
27 function claim(address token) external;
```

CVF-40 FIXED

- **Category** Suboptimal

- **Source** AcrossBridgeAdapter.sol

Description As “feePct” is rounded down, the condition could be true even when fee is actually too high.

Recommendation Check that $\text{fee} * \text{MAX_FEE_CAP_PCT} < \text{amount} * \text{feeCapPct}$ instead.

```
72 uint256 feePct = acrossParams.fee.mulDiv(MAX_FEE_CAP_PCT,  
    ↪ instruction.amount);
```

```
76 require(feePct < feeCapPct, FeeTooHigh(acrossParams.fee));
```

CVF-41 FIXED

- **Category** Suboptimal

- **Source** AcrossBridgeAdapter.sol

Description As “slippageDeltaPct” is rounded down, the condition could be true even when slippage is actually too high.

Recommendation Check that $\text{slippageDelta} * \text{MAX_FEE_CAP_PCT} < \text{amount} * \text{feeCapPct}$ instead.

```
74 uint256 slippageDeltaPct = slippageDelta.mulDiv(MAX_FEE_CAP_PCT,  
    ↪ instruction.amount);
```

```
77 require(  
    slippageDeltaPct < feeCapPct,
```

CVF-42 FIXED

- **Category** Procedural

- **Source** BridgeAdapter.sol

Recommendation Unchained initializers should be used here.

```
27 __AccessControl_init();  
   __ReentrancyGuard_init();
```

CVF-43 FIXED

- **Category** Suboptimal

- **Source** StrategyTemplate.sol

Description This allows the same token to be stored several times.

Recommendation Require token addresses to go in ascending order to prevent duplicates.

```
74 _inputTokens.push(__inputTokens[i]);
```

```
78 _outputTokens.push(__outputTokens[i]);
```

CVF-44 FIXED

- **Category** Procedural

- **Source** StrategyTemplate.sol

Description These function are very similar.

Recommendation Extract common parts into a utility function to avoid code duplication.

```
88 function enter(
```

```
111 function reenterAfterEmergencyExit(uint256 minNavDelta) public  
    ↪ payable override onlyEmergencyManager {
```

CVF-45 FIXED

- **Category** Procedural

- **Source** Container.sol

Description The returned value is ignored.

Recommendation Explicitly require the return values to be "true" or use the "safeApprove" function.

```
67 IERC20(token).approve(swapRouter, type(uint256).max);
```

```
74 IERC20(token).approve(swapRouter, 0);
```

```
160 IERC20(token).approve(addr, 0);
```

```
168 IERC20(token).approve(addr, type(uint256).max);
```

CVF-46 FIXED

- **Category** Unclear behavior

- **Source** Container.sol

Recommendation This function should delete the dust threshold potentially set for the token being blacklisted.

```
71 function blacklistToken(address token) external onlyRole(  
    ↪ TOKEN_MANAGER_ROLE) {
```

CVF-47 INFO

- **Category** Procedural

- **Source** Container.sol

Description This check should be removed because approvals are granted to the whitelisted tokens even when the swap router address is zero.

Recommendation Remove this check and always drop approvals from the whitelisted tokens.

Client Comment *Dropping approvals to the zero address is impossible - the OpenZeppelin ERC20 implementation reverts when attempting to set approval for the zero address. Therefore, the check `previousSwapRouter != address(0)` must remain to prevent unintended reverts and ensure safe execution.*

```
131 if (previousSwapRouter != address(0)) {  
    _dropApprovesFromWhitelistedTokens(previousSwapRouter);
```

CVF-48 FIXED

- **Category** Unclear behavior
- **Source** StrategyContainer.sol

Description It is not guaranteed that the tokens returned here are the same tokens that were approved when the strategy was added.

Recommendation Implement some way to guarantee the same list of input tokens is used when adding and when removing a strategy.

```
133 address[] memory inputTokens = IStrategyTemplate(strategy).  
    ↪ inputTokens();
```

```
136 for (uint256 i = 0; i < length; ++i) {  
    IERC20(inputTokens[i]).approve(strategy, 0);
```

CVF-49 INFO

- **Category** Suboptimal
- **Source** StrategyContainer.sol

Description This means very coarse precision of the share. The minimum lot to be exited is 0.01% of all the assets in the strategy, which could be quite a large sum.

Recommendation Use higher denominator, such as 1e18.

Client Comment *The current precision (BPS, i.e., 0.01%) is intentionally chosen and aligned with the system's design requirements. Finer fractional exits are not needed for operational or UX purposes.*

```
278 require(share <= BPS, Errors.IncorrectAmount());
```

CVF-50 FIXED

- **Category** Unclear behavior
- **Source** ContainerAgent.sol

Description There are no check to ensure "bridgeAdapters" and "bridgeInstructions" arrays are of the same length. In case the "bridgeAdapters" arrays is longer, extra elements are silently ignored.

Recommendation Add proper length check.

```
116 address[] calldata bridgeAdapters,  
    IBridgeAdapter.BridgeInstruction[] calldata bridgeInstructions
```

```
166 address[] calldata bridgeAdapters,  
    IBridgeAdapter.BridgeInstruction[] calldata bridgeInstructions
```

CVF-51 FIXED

- **Category** Procedural

- **Source** ContainerAgent.sol

Recommendation This TODO should be resolved or removed.

```
175 // TODO: Wrap batch processing finishing in an internal method
```

CVF-52 FIXED

- **Category** Suboptimal

- **Source** ContainerAgent.sol

Description These checks are redundant, as there arrays are allocated a few lines above.

Recommendation Perform all necessary checks before allocating the arrays.

```
189 require(tokens.length > 0, Errors.ZeroAmount());
190 require(minAmounts.length == tokens.length, Errors.
    ↳ ArrayLengthMismatch());
```

CVF-53 FIXED

- **Category** Procedural

- **Source** ContainerAgent.sol

Recommendation This TODO should be resolved or removed.

```
216 // TODO: Check if length == 0 zero requires extra handling
```

CVF-54 FIXED

- **Category** Unclear behavior

- **Source** MessageRouter.sol

Description There is no range check for this argument.

Recommendation Add an appropriate check.

```
35 uint256 maxCacheSize
```

CVF-55 INFO

- **Category** Bad datatype
- **Source** MessageRouter.sol

Recommendation The argument type should be "IMessageAdapter".

Client Comment *An interface type is not required here because the contract does not perform any direct calls to the adapter. We only store the address for whitelist/blacklist management, so using a plain address type avoids unnecessary coupling and keeps the function flexible for future adapter implementations.*

```
88 function whitelistMessageAdapter(address adapter) external onlyRole(  
    ↪ GOVERNANCE_ROLE) {  
  
94 function blacklistMessageAdapter(address adapter) external onlyRole(  
    ↪ GOVERNANCE_ROLE) {
```

CVF-56 FIXED

- **Category** Unclear behavior
- **Source** PriceOracle.sol

Description There is no check to ensure "feed" is not zero, while zero feed has a special value: not set.

Recommendation Add an appropriate check.

```
37 function setPriceFeed(address token, address feed) external onlyRole  
    ↪ (GOVERNANCE_ROLE) {
```

CVF-57 FIXED

- **Category** Suboptimal
- **Source** PriceOracle.sol

Description Calculating "10 ** decimals" every time is inefficient, taking into account, that for a particular price feed the "decimals" property most probably never changes.

Recommendation Precompute "10 ** decimals" when possible.

```
55 (uint256 price, uint256 decimals) = getPrice(token);  
    return amount.mulDiv(price, 10 ** decimals);  
  
60 (uint256 price, uint256 decimals) = getPrice(token);  
    uint256 value = amount.mulDiv(price, 10 ** decimals);
```

CVF-58 INFO

- **Category** Suboptimal
- **Source** ReshufflingGateway.sol

Recommendation Unchained initializers should be used here.

Client Comment *Switching to unchained initializers would not provide any functional or gas-related benefit here.*

```
62  __AccessControl_init();
    __ReentrancyGuard_init();
```

CVF-59 FIXED

- **Category** Suboptimal
- **Source** ReshufflingGateway.sol

Recommendation This function should return the amount claimed.

```
93  function claimBridge(address bridgeAdapter, address token) external
    ↪ nonReentrant {
```

CVF-60 FIXED

- **Category** Procedural
- **Source** ReshufflingGateway.sol

Recommendation This TODO should be either resolved or removed.

```
114  // todo: think about approve
```

```
154  // todo: think about approve
```

```
170  // todo: add container type check
```

CVF-61 FIXED

- **Category** Unclear behavior
- **Source** ReshufflingGateway.sol

Description The returned value is ignored here.

Recommendation Explicitly require the returned value to be true, or use the "safeApprove" function.

```
155  IERC20(instructions[i].token).approve(bridgeAdapters[i],
    ↪ instructions[i].amount);
```

CVF-62 INFO

- **Category** Suboptimal

- **Source** SwapRouter.sol

Recommendation Unchained initializers should be used here.

Client Comment *Switching to unchained initializers would not provide any functional or gas-related benefit here.*

```
31 __AccessControl_init();  
   __ReentrancyGuard_init();
```

CVF-63 FIXED

- **Category** Procedural

- **Source** SwapRouter.sol

Recommendation This TODO should be either resolved or removed.

```
64 SlippageNotMet(amountOutBefore, amountOutAfter, instruction.  
   ↪ minAmountOut) // todo: inline with other slippage checks
```

CVF-64 INFO

- **Category** Suboptimal

- **Source** Vault.sol

Recommendation Unchained initializers should be used here.

Client Comment *Switching to unchained initializers would not provide any functional or gas-related benefit here.*

```
111 __AccessControl_init();  
    __ERC20_init(_name, _symbol);  
    __ReentrancyGuard_init();
```

CVF-65 FIXED

- **Category** Procedural

- **Source** Vault.sol

Description These checks are inconsistent with each other. It is not possible to set the maximum deposit amount larger than then maximum deposit batch size, but it is still possible to set the maximum deposit batch size smaller than the maximum deposit amount.

Recommendation Make the checks consistent. Probably, it should be allowed to set the maximum deposit amount larger than the maximum deposit batch size.

```
179 function _setMaxDepositAmount(uint256 _maxDepositAmount) internal {
180     require(
        _maxDepositAmount > minDepositAmount && _maxDepositAmount <=
            ↪ maxDepositBatchSize,
        Errors.IncorrectAmount()
    );
```

```
194 function _setMaxDepositBatchSize(uint256 _maxDepositBatchSize)
    ↪ internal {
    require(_maxDepositBatchSize > minDepositBatchSize, Errors.
        ↪ IncorrectAmount());
```

CVF-66 FIXED

- **Category** Procedural

- **Source** Vault.sol

Description These checks are inconsistent with each other. When comparing the maximum deposit amount with the maximum deposit batch size a non-strict "<=" is used, while when comparing the minimum deposit batch size with the minimum deposit amount, a strict ">" is used.

Recommendation Use non-strict operators in both cases.

```
180 require(
    _maxDepositAmount > minDepositAmount && _maxDepositAmount <=
        ↪ maxDepositBatchSize,
    Errors.IncorrectAmount()
);
```

```
201 require(
    _minDepositBatchSize > minDepositAmount && _minDepositBatchSize
        ↪ < maxDepositBatchSize,
    Errors.IncorrectAmount()
);
```

CVF-67 FIXED

- **Category** Procedural

- **Source** Vault.sol

Description These checks are inconsistent with each other. It is impossible to set the minimum deposit batch size smaller than the minimum deposit amount, but it is still possible to set the minimum deposit amount larger than the minimum deposit batch size.

Recommendation Make the checks consistent. Probably, it should be allowed to set the minimum deposit batch size smaller than the minimum deposit amount.

```
188 function _setMinDepositAmount(uint256 _minDepositAmount) internal {  
    require(_minDepositAmount > 0 && _minDepositAmount <  
        ↪ maxDepositAmount, Errors.IncorrectAmount());
```

```
200 function _setMinDepositBatchSize(uint256 _minDepositBatchSize)  
    ↪ internal {  
    require(  
        _minDepositBatchSize > minDepositAmount &&  
        ↪ _minDepositBatchSize < maxDepositBatchSize,  
        Errors.IncorrectAmount()  
    );
```

CVF-68 FIXED

- **Category** Suboptimal

- **Source** Vault.sol

Recommendation Should be ">=" and "<=" instead of ">" and "<".

```
210 require(_minWithdrawBatchRatio > 0 && _minWithdrawBatchRatio <  
    ↪ MAX_BPS, Errors.IncorrectAmount());
```

CVF-69 INFO

- **Category** Procedural

- **Source** Vault.sol

Recommendation This check should be done before adding the container. In which a case "<" should be used instead of "<=".

Client Comment Due to how the OpenZeppelin EnumerableSet library works, the element is added before we can determine whether the insertion was successful. Therefore, the check must be performed after adding the container. Additionally, verifying the length post-insertion avoids unnecessary pre-checks and results in better gas efficiency.

```
234 uint256 length = _containers.length();  
    require(length <= MAX_CONTAINERS, MaxContainersReached());
```

CVF-70 FIXED

- **Category** Suboptimal

- **Source** Vault.sol

Description The “mulDiv” function rounds down, which means that each claim will make the shares to deposit amount ratio slightly better for those who will claim after. This is unfair.

Recommendation Use the initial total shares and initial total notion values for all claims, rather than remaining total shares and remaining total notion.

```
328 vars.sharesToClaim = vars.depositAmount.mulDiv(vars.batchTotalShares  
    ↪ , vars.batchTotalNotion);
```

```
332 depositBatchTotalShares[batchId] = vars.batchTotalShares - vars.  
    ↪ sharesToClaim;
```

CVF-71 FIXED

- **Category** Suboptimal

- **Source** Vault.sol

Description The “mulDiv” function rounds down, which means that each claim will make the remaining notion to deposit amount ratio slightly better for those who will claim after. This is unfair.

Recommendation Use the initial notion remainder and initial total notion values for all claims, rather than remaining notion remainder and remaining total notion.

```
335 vars.notionToClaim = vars.depositAmount.mulDiv(vars.  
    ↪ batchNotionRemainder, vars.batchTotalNotion);
```

```
337 totalUnclaimedNotionRemainder -= vars.notionToClaim;
```

CVF-72 INFO

- **Category** Suboptimal

- **Source** Vault.sol

Description It seems weird to specify the shares precent to be redeemed, rather than the shares amount.

Recommendation Allow the user to specify the shares amount to be redeemed.

Client Comment *The contract intentionally uses a percentage-based withdrawal model rather than a raw share amount. This simplifies the UX, guarantees proportional withdrawal across strategies, and prevents rounding or dust issues. Allowing users to specify arbitrary share amounts contradicts the current design and is intended by design.*

```
352 function withdraw(uint256 sharesPercent) external nonReentrant  
    ↪ notInReshufflingMode {
```

CVF-73 FIXED

- **Category** Suboptimal

- **Source** Vault.sol

Description The “mulDiv” function rounds down, which means that each withdrawal will make the remaining notion per share ratio slightly better for those who will claim after. This is unfair.

Recommendation Use the initial total notion and initial total shares values for all withdrawals, rather than remaining total notion and remaining total shares.

```
378 vars.notionToClaim = vars.withdrawnShares.mulDiv(vars.  
    ↪ batchTotalNotion, vars.batchTotalShares);
```

```
383 withdrawBatchTotalNotion[batchId] = vars.batchTotalNotion - vars.  
    ↪ notionToClaim;
```

CVF-74 FIXED

- **Category** Procedural

- **Source** Vault.sol

Recommendation This TODO should be resolved or removed.

```
394 // TODO: Implement PAUSE LOGIC
```

CVF-75 FIXED

- **Category** Suboptimal

- **Source** Vault.sol

Description This means that the last container will get all the accumulated rounding errors, which could lead to a systematic bias towards the latest container.

Recommendation Calculate the container amount using the remaining deposit amount and the remaining total weight. This would distribute rounding errors from earlier container evenly among later containers.

```
438 if (i == vars.lastContainerIndex) {  
    // NOTE: Last container takes all division dust  
440    vars.containerAmount = vars.totalBatchDepositAmount - vars.  
        ↪ distributedNotion;  
} else {  
    vars.containerAmount = vars.totalBatchDepositAmount.mulDiv(  
        ↪ containerWeight, MAX_BPS);  
}
```



8 Recommendations

CVF-76 INFO

- **Category** Procedural
- **Source** OneInchAdapter.sol

Recommendation Consider specifying as "^{0.8.0}" unless there is something special regarding this particular version. See also: UniswapV3Adapter.sol, UniversalAdapter.sol, CompoundV3.sol, RingCacheLibrary.sol, Errors.sol, Common.sol, EnumerableAddressSetExtended.sol, AddressArrayPackingLib.sol, ContainerMessagePackingLib.sol, DepositRequestLib.sol, DepositResponseLib.sol, MessageCodec.sol, WithdrawalRequestLib.sol, WithdrawalResponseLib.sol, IAccessControlManager.sol, IUniversalAdapter.sol, IVault.sol, IBridgeAdapter.sol, AcrossBridgeAdapter.sol, BridgeAdapter.sol, StrategyTemplate.sol, Collector.sol, Container.sol, StrategyContainer.sol, CrossChainContainer.sol, ContainerAgent.sol, ContainerLocal.sol, ContainerPrincipal.sol, MessageRouter.sol, PriceOracle.sol, ReshufflingGateway.sol, SwapRouter.sol, Vault.sol, IContainerAgent.sol, IContainerLocal.sol, ISwapRouter.sol, ICrossChainContainer.sol, IContainerPrincipal.sol, IMessageAdapter.sol, IMessageReceiver.sol, IOneInchAdapter.sol, IPriceOracle.sol, IStrategyContainer.sol, IReshufflingGateway.sol, IMessageRouter.sol, IStrategyTemplate.sol, ISwapAdapter.sol.

```
2 pragma solidity ^0.8.28;
```

CVF-77 INFO

- **Category** Procedural
- **Source** OneInchAdapter.sol

Recommendation Consider importing directly from the 1inch repo.

```
6 import {IOneInchV6} from "../dependencies/interfaces/oneInch/
   ↪ IOneInchV6.sol";
```

CVF-78 INFO

- **Category** Bad datatype
- **Source** OneInchAdapter.sol

Recommendation The type for this variable should be "IOneInchV6".

```
15 address public immutable oneInchRouter;
```

CVF-79 INFO

- **Category** Bad datatype
- **Source** OneInchAdapter.sol

Recommendation The argument type should be "IOneInchV6".

```
17 constructor(address _oneInchRouter) {
```

CVF-80 INFO

- **Category** Bad datatype
- **Source** OneInchAdapter.sol

Recommendation The type for these arguments should be "IERC20".

```
22 address tokenIn,  
address tokenOut,
```

CVF-81 INFO

- **Category** Procedural
- **Source** UniswapV3Adapter.sol

Recommendation Consider importing this file directly from the Uniswap repo.

```
11 import {IUniswapV3Router} from "../dependencies/interfaces/uniswapV3  
↪ /IUniswapV3.sol";
```

CVF-82 INFO

- **Category** Bad datatype
- **Source** UniswapV3Adapter.sol

Recommendation The type for this variable should be "IUniswapV3Router".

```
19 address public immutable uniswapV3Router;
```

CVF-83 INFO

- **Category** Bad datatype
- **Source** UniswapV3Adapter.sol

Recommendation The type for the "_uniswapV3Router" argument should be "IUniswapV3Router".

```
21 constructor(address defaultAdmin, address _uniswapV3Router) {
```

CVF-84 INFO

- **Category** Bad datatype
- **Source** UniswapV3Adapter.sol

Recommendation The type for the token arguments should be "IERC20".

```
31 function whitelistPath(address tokenIn, address tokenOut, uint24 fee  
    ↪ ) external onlyWhitelistManager {
```

```
36 function blacklistPath(address tokenIn, address tokenOut, uint24 fee  
    ↪ ) external onlyWhitelistManager {
```

CVF-85 FIXED

- **Category** Unclear behavior
- **Source** UniswapV3Adapter.sol

Description These functions should emit some events.

```
31 function whitelistPath(address tokenIn, address tokenOut, uint24 fee  
    ↪ ) external onlyWhitelistManager {
```

```
36 function blacklistPath(address tokenIn, address tokenOut, uint24 fee  
    ↪ ) external onlyWhitelistManager {
```

CVF-86 FIXED

- **Category** Bad naming
- **Source** UniswapV3Adapter.sol

Description Despite the name this function doesn't actually adds a path to the black list, but just removes it from the while list. Actually, there is no such thin as the black list in this contract.

Recommendation Choose another name.

```
36 function blacklistPath(address tokenIn, address tokenOut, uint24 fee  
    ↪ ) external onlyWhitelistManager {
```

CVF-87 INFO

- **Category** Bad datatype
- **Source** UniswapV3Adapter.sol

Recommendation The type for these fields should be "IERC20".

```
42 address tokenIn,  
    address tokenOut,
```



CVF-88 INFO

- **Category** Bad datatype
- **Source** UniversalAdapter.sol

Description The semantics of the keys in this mapping is unclear.

Recommendation Give descriptive names to the keys and/or explain in a documentation comment.

```
21 mapping(address => mapping(address => SwapInfo)) public swapInfos;
```

CVF-89 INFO

- **Category** Bad datatype
- **Source** UniversalAdapter.sol

Recommendation The keys type for this mapping should be "IERC20".

```
21 mapping(address => mapping(address => SwapInfo)) public swapInfos;
```

CVF-90 INFO

- **Category** Bad datatype
- **Source** UniversalAdapter.sol

Recommendation The type for the arguments should be "IERC20".

```
32 function isSwapSupported(address tokenIn, address tokenOut) external  
    ↪ view override returns (bool) {
```

CVF-91 INFO

- **Category** Bad datatype
- **Source** UniversalAdapter.sol

Recommendation The type for these arguments should be "IERC20".

```
37 address tokenIn,  
    address tokenOut,
```

```
59 address fromToken,  
60 address toToken,
```

CVF-92 INFO

- **Category** Bad datatype
- **Source** UniversalAdapter.sol

Recommendation The type for this argument should be more specific.

```
61 address router,
```

CVF-93 INFO

- **Category** Bad datatype
- **Source** UniversalAdapter.sol

Recommendation The type for the token arguments should be "IERC20".

```
69 function _executeSwap(SwapInfo memory swapInfo, address tokenIn,  
    ↪ uint256 amountIn) private {
```

CVF-94 INFO

- **Category** Procedural
- **Source** CompoundV3.sol

Recommendation Consider importing these files directly from the compound's repo.

```
10 import {ICometV3, IRewards} from "../dependencies/interfaces/  
    ↪ compoundV3/ICompoundV3.sol";  
import {IUniswapV3Router} from "../dependencies/interfaces/uniswapV3  
    ↪ /IUniswapV3.sol";
```

CVF-95 FIXED

- **Category** Unclear behavior
- **Source** CompoundV3.sol

Description The purpose of this constant is unclear.

Recommendation Use a more descriptive name or explain in a documentation comment or just remove this constant.

```
17 bytes32 private constant COMPOUND_V3 = bytes32(0);
```

CVF-96 FIXED

- **Category** Bad datatype
- **Source** CompoundV3.sol

Recommendation The type for this variable should be "IComeV3".

```
20 address public cometV3;
```

CVF-97 FIXED

- **Category** Bad datatype
- **Source** CompoundV3.sol

Recommendation The type for this variable should be "IERC20".

```
21 address public cometV3Underlying;
```

CVF-98 FIXED

- **Category** Bad datatype
- **Source** CompoundV3.sol

Recommendation The type for this variable should be "IRewards".

```
22 address public rewards;
```

CVF-99 FIXED

- **Category** Bad datatype
- **Source** CompoundV3.sol

Recommendation The type for the "_cometV3" argument should be "ICometV3".

```
35 function __CompoundV3_init(address _cometV3, address _rewards,  
    ↪ address _containerAgent) internal onlyInitializing {
```

CVF-100 FIXED

- **Category** Bad datatype
- **Source** CompoundV3.sol

Recommendation The type for the "_rewards" argument should be "IRewards".

```
35 function __CompoundV3_init(address _cometV3, address _rewards,  
    ↪ address _containerAgent) internal onlyInitializing {
```

CVF-101 FIXED

- **Category** Bad datatype
- **Source** CompoundV3.sol

Recommendation The type for the “_containerAgent” argument should be “IContainerAgent”.

```
35 function __CompoundV3_init(address _cometV3, address _rewards,  
    ↪ address _containerAgent) internal onlyInitializing {
```

CVF-102 FIXED

- **Category** Suboptimal
- **Source** CompoundV3.sol

Description This check is redundant, as it is anyway possible to pass a dead comet V3 address.

Recommendation Remove this check.

```
36 require(_cometV3 != address(0), Errors.ZeroAddress());
```

CVF-103 FIXED

- **Category** Bad datatype
- **Source** CompoundV3.sol

Recommendation The argument type should be “ISwapRouter”.

```
76 function _claimAndSwapRewards(address _swapRouter) internal override  
    ↪ {
```

CVF-104 FIXED

- **Category** Bad naming
- **Source** CompoundV3.sol

Description The semantics of the “true” value passed as the last argument is unclear.

Recommendation Give the argument name as a comment next to the value.

```
77 IRewards(rewards).claim(cometV3, address(this), true);
```

CVF-105 FIXED

- **Category** Bad datatype
- **Source** CompoundV3.sol

Recommendation The argument type should be "IERC20".

```
82 function _swapRewards(address token) internal virtual;
```

CVF-106 FIXED

- **Category** Procedural
- **Source** CompoundV3.sol

Recommendation This contract should be moved into a separate file named "CompoundV3ArbitrumUSDC".

```
95 contract CompoundV3ArbitrumUSDC is CompoundV3 {
```

CVF-107 FIXED

- **Category** Procedural
- **Source** CompoundV3.sol

Description Hardcoding mainnet addresses is a bad practice as it makes testing harder.

Recommendation Pass addresses as constructor arguments and store them into immutable variables.

```
96 address private constant COMET_V3_USDC = 0
    ↪ x9c4ec768c28520B50860ea7a15bd7213a9fF58bf;
address private constant REWARDS = 0
    ↪ x88730d254A2f7e6AC8388c3198aFd694bA9f7fae;
address private constant UNISWAP_V3_ROUTER = 0
    ↪ xE592427A0AEce92De3Edee1F18E0157C05861564;
address private constant WETH = 0
    ↪ x82aF49447D8a07e3bd95BD0d56f35241523fBab1;
```

CVF-108 FIXED

- **Category** Bad datatype
- **Source** CompoundV3.sol

Recommendation The type for this constant should be "ICometV3".

```
96 address private constant COMET_V3_USDC = 0
    ↪ x9c4ec768c28520B50860ea7a15bd7213a9fF58bf;
```

CVF-109 FIXED

- **Category** Bad datatype
- **Source** CompoundV3.sol

Recommendation The type for this constant should be "IRewards".

```
97 address private constant REWARDS = 0  
    ↪ x88730d254A2f7e6AC8388c3198aFd694bA9f7fae;
```

CVF-110 FIXED

- **Category** Bad datatype
- **Source** CompoundV3.sol

Recommendation The type for this constant should be "IUniswapV3Router".

```
98 address private constant UNISWAP_V3_ROUTER = 0  
    ↪ xE592427A0AEce92De3Edee1F18E0157C05861564;
```

CVF-111 FIXED

- **Category** Bad datatype
- **Source** CompoundV3.sol

Recommendation The type for this constant should be "IWETH9".

```
99 address private constant WETH = 0  
    ↪ x82aF49447D8a07e3bd95BD0d56f35241523fBab1;
```

CVF-112 FIXED

- **Category** Bad datatype
- **Source** CompoundV3.sol

Recommendation The argument type should be "IContainerAgent".

```
101 function initialize(address _containerAgent) public override  
    ↪ initializer {
```

CVF-113 FIXED

- **Category** Bad datatype

- **Source** CompoundV3.sol

Recommendation This value should be a named constant.

```
115 rewardToWethParams.fee = uint24(3000);
```

```
127 wethToUnderlyingParams.fee = uint24(500);
```

CVF-114 FIXED

- **Category** Procedural

- **Source** RingCacheLibrary.sol

Recommendation This error could be made more useful by adding certain parameters into it.

```
20 error InvalidSize();
```

CVF-115 INFO

- **Category** Procedural

- **Source** Errors.sol

Recommendation These errors could be made more useful by adding certain parameters into them.

```
7 error Unauthorized();  
error OnlyAgent();
```

```
12 error AlreadyBlacklisted();  
error AlreadyInitialized();  
error AlreadyWhitelisted();
```

```
16 error NotImplemented();
```

```
21 error ArrayLengthMismatch();  
error IncorrectAmount();  
error IncorrectContainerStatus();  
error IncorrectInput();  
error InvalidDataLength();
```

```
29 error NonZeroAmount();
```

CVF-116 INFO

- **Category** Bad datatype
- **Source** Errors.sol

Recommendation The type for the "token" parameter should be "IERC20".

```
33 error NotEnoughTokens(address token, uint256 amount);
```

CVF-117 INFO

- **Category** Bad datatype
- **Source** Common.sol

Recommendation The type for the "token" argument should be "IERC20".

```
15 function toUnifiedDecimalsUint8(address token, uint256 amount)
    ↳ internal view returns (uint256) {
```

```
24 function fromUnifiedDecimalsUint8(address token, uint256 amount)
    ↳ internal view returns (uint256) {
```

CVF-118 INFO

- **Category** Procedural
- **Source** Common.sol

Description In ERC20 the "decimals" property is used by UI to render token amounts in a human -readable way. Using this property in smart contracts is discouraged.

Recommendation Treat all token amounts as integers.

```
16 uint8 decimals = IERC20Metadata(token).decimals();
```

```
25 uint8 decimals = IERC20Metadata(token).decimals();
```

CVF-119 INFO

- **Category** Suboptimal
- **Source** Common.sol

Recommendation With numerator always equal to 1, there is no need to the "muldiv" function here. Just use plain division instead.

```
30 return amount.mulDiv(ONE, 10 ** (UNIFIED_DECIMALS - decimals), Math.
    ↳ Rounding.Floor);
```

CVF-120 FIXED

- **Category** Suboptimal

- **Source** AddressArrayPackingLib.sol

Description This function is inefficient.

Recommendation Consider implementing like this: <https://gist.github.com/3sG-gpQ8H/2e62fecf69411420d1e27a6750a1f123>

```
7 function packAddresses(address[] memory addresses) internal pure  
    ↪ returns (bytes memory) {
```

CVF-121 FIXED

- **Category** Suboptimal

- **Source** AddressArrayPackingLib.sol

Recommendation This line could be optimized by maintaining a pointer to the current address and incrementing it by 20 on each iteration.

```
28 addr := mload(add(add(data, 20), mul(i, 20)))
```

CVF-122 FIXED

- **Category** Suboptimal

- **Source**
ContainerMessagePackingLib.sol

Recommendation Consider arranging these constants so their values come in sequence.

```
14 uint8 constant DEPOSIT_REQUEST_TYPE = 0;  
    uint8 constant WITHDRAWAL_REQUEST_TYPE = 2;  
    uint8 constant DEPOSIT_RESPONSE_TYPE = 1;  
    uint8 constant WITHDRAWAL_RESPONSE_TYPE = 3;
```

CVF-123 FIXED

- **Category** Procedural

- **Source**
ContainerMessagePackingLib.sol

Recommendation This comment should be translated into English or removed.

```
34 mstore(payload, sub(mload(data), 1)) // Устанавливаем длину data
```

CVF-124 FIXED

- **Category** Procedural

- **Source**

ContainerMessagePackingLib.sol

Recommendation This check could be done earlier like this: `require (data.length <= MAXIMUM_PAYLOAD_LENGTH);`

```
36 require(payload.length < MAXIMUM_PAYLOAD_LENGTH, Errors.  
    ↪ InvalidDataLength());
```

CVF-125 FIXED

- **Category** Suboptimal

- **Source**

ContainerMessagePackingLib.sol

Recommendation This condition could be replaced with a single check: `type_ < 4`

```
42 (type_ != DEPOSIT_REQUEST_TYPE) &&  
    (type_ != WITHDRAWAL_REQUEST_TYPE) &&  
    (type_ != DEPOSIT_RESPONSE_TYPE) &&  
    (type_ != WITHDRAWAL_RESPONSE_TYPE)
```

CVF-126 FIXED

- **Category** Suboptimal

- **Source** DepositRequestLib.sol

Recommendation It would be more efficient to use a single array of structs with two fields, rather than two parallel arrays.

```
15 address[] tokens;  
    uint256[] amounts;
```

CVF-127 FIXED

- **Category** Suboptimal

- **Source** DepositResponseLib.sol

Recommendation It would be more efficient to use a single array of structs with two fields, rather than two parallel arrays.

```
15 address[] tokens;  
    uint256[] amounts;
```

CVF-128 FIXED

- **Category** Procedural

- **Source** MessageCodec.sol

Recommendation Consider using the identity precompiled contract (0x04) to copy memory.

```
33 assembly {  
    // Copy remaining bytes after nonce and path  
    let messageStart := add(add(message, 64), 32)  
    let messageDataStart := add(messageData, 32)  
    for {  
        let i := 0  
    } lt(i, messageLength) {  
40     i := add(i, 32)  
    } {  
        mstore(add(messageDataStart, i), mload(add(messageStart, i))  
        ↪ )  
    }  
}
```

CVF-129 INFO

- **Category** Suboptimal

- **Source** WithdrawalResponseLib.sol

Recommendation It would be more efficient to use a single array of structs with two fields rather than two parallel arrays.

```
15 address[] tokens;  
    uint256[] amounts;
```

CVF-130 INFO

- **Category** Bad naming

- **Source** IContainer.sol

Recommendation Events are usually named via nouns, such as "SwapRouter" or "WhitelistedTokenDustThreshold".

```
11 event SwapRouterUpdated(address indexed previousSwapRouter, address  
    ↪ indexed newSwapRouter);  
event WhitelistedTokenDustThresholdUpdated(address indexed token,  
    ↪ uint256 threshold);
```

CVF-131 INFO

- **Category** Suboptimal
- **Source** IContainer.sol

Recommendation The previous value parameter is redundant, as its value could be derived from the previous event.

```
11 event SwapRouterUpdated(address indexed previousSwapRouter, address  
    ↳ indexed newSwapRouter);
```

CVF-132 INFO

- **Category** Bad datatype
- **Source** IContainer.sol

Recommendation The type for the parameters should be "ISwapRouter".

```
11 event SwapRouterUpdated(address indexed previousSwapRouter, address  
    ↳ indexed newSwapRouter);
```

CVF-133 INFO

- **Category** Bad datatype
- **Source** IContainer.sol

Recommendation The type for the "token" parameter should be "IERC20".

```
12 event WhitelistedTokenDustThresholdUpdated(address indexed token,  
    ↳ uint256 threshold);
```

CVF-134 INFO

- **Category** Procedural
- **Source** IContainer.sol

Recommendation These errors could be made more useful by adding certain parameters into them.

```
14 error AlreadyWhitelistedToken();
```

```
17 error WhitelistedTokensOnBalance();
```

CVF-135 INFO

- **Category** Bad datatype
- **Source** IContainer.sol

Recommendation The type for the "token" parameters should be "IERC20".

```
15 error NotWhitelistedToken(address token);
```

CVF-136 INFO

- **Category** Procedural
- **Source** IContainer.sol

Description This event is placed between among errors which makes code harder to read.

Recommendation Move it next to other events.

```
16 event TokenWhitelistUpdated(address indexed token, bool  
    ↪ isWhitelisted);
```

CVF-137 INFO

- **Category** Bad datatype
- **Source** IContainer.sol

Recommendation The argument type should be "IERC20".

```
20 function isTokenWhitelisted(address token) external view returns (  
    ↪ bool);
```

CVF-138 INFO

- **Category** Bad datatype
- **Source** IContainer.sol

Recommendation The result type should be "ISwapRouter".

```
21 function swapRouter() external view returns (address);
```

CVF-139 INFO

- **Category** Bad datatype
- **Source** IContainerAgent.sol

Recommendation The type for this field should be "IERC20[]".

```
17 address[] tokens;
```

CVF-140 INFO

- **Category** Suboptimal
- **Source** IContainerAgent.sol

Recommendation It would be more efficient to use a single array of structs with two fields, rather than two parallel arrays.

```
17 address[] tokens;  
   uint256[] minAmounts;
```

CVF-141 INFO

- **Category** Bad naming
- **Source** IContainerAgent.sol

Recommendation Events are usually named via nouns, such as "AllStrategiesEntering" or "AllStrategiesExit".

```
24 event AllStrategiesEntered();  
   event AllStrategiesExited();  
   event DepositReported(uint256 nav0, uint256 nav1);  
   event WithdrawalReported(uint256 shares);  
   event DepositRequestReceived(uint256 claimCounter);  
   event WithdrawalRequestReceived(uint256 shares);
```

CVF-142 INFO

- **Category** Bad datatype
- **Source** IContainerAgent.sol

Recommendation The type for the "bridgeAdapter" should be "IBridgeAdapter".

```
32 function claim(address bridgeAdapter, address token) external;
```

CVF-143 INFO

- **Category** Bad datatype
- **Source** IContainerAgent.sol

Recommendation The type for the "token" argument should be "IToken".

```
32 function claim(address bridgeAdapter, address token) external;
```

CVF-144 INFO

- **Category** Bad datatype

- **Source** IContainerAgent.sol

Recommendation The type for the "bridgeAdapters" argument should be "IBridgeAdapter[]".

```
34 function claimMultiple(address[] memory bridgeAdapters, address[]  
    ↪ memory tokens) external;
```

CVF-145 INFO

- **Category** Bad datatype

- **Source** IContainerAgent.sol

Recommendation The type for the "tokens" argument should be "IERC20[]".

```
34 function claimMultiple(address[] memory bridgeAdapters, address[]  
    ↪ memory tokens) external;
```

CVF-146 INFO

- **Category** Suboptimal

- **Source** IContainerAgent.sol

Recommendation It would be more efficient to pass a single array of structs with two fields, rather than two parallel arrays. This would also make the length check unnecessary.

```
34 function claimMultiple(address[] memory bridgeAdapters, address[]  
    ↪ memory tokens) external;
```

CVF-147 INFO

- **Category** Bad naming

- **Source** IContainerLocal.sol

Recommendation Events are usually named via nouns, such as "AllStrategiesEntering" or "AllStrategiesExit".

```
17 event AllStrategiesEntered();  
    event AllStrategiesExited();
```

CVF-148 INFO

- **Category** Bad datatype
- **Source** IContainerLocal.sol

Recommendation The type for the "strategy" argument should be more specific.

```
30 function enterStrategy(address strategy, uint256[] calldata  
    ↪ inputAmounts, uint256 minNavDelta) external;
```

```
38 function exitStrategy(address strategy, uint256 minNavDelta)  
    ↪ external;
```

CVF-149 INFO

- **Category** Bad datatype
- **Source** IContainerLocal.sol

Recommendation The type for the "strategies" argument should be more specific.

```
33 address[] calldata strategies,
```

```
40 function exitStrategyMultiple(address[] calldata strategies, uint256  
    ↪ [] calldata minNavDelta) external;
```

CVF-150 INFO

- **Category** Suboptimal
- **Source** IContainerLocal.sol

Recommendation It would be more efficient to pass a single array of structs with three fields, rather than three parallel arrays. This would also make the length check unnecessary.

```
33 address[] calldata strategies,  
    uint256[][] calldata inputAmounts,  
    uint256[] calldata minNavDelta
```

CVF-151 INFO

- **Category** Suboptimal
- **Source** IContainerLocal.sol

Recommendation It would be more efficient to pass a single array of structs with two fields, rather than two parallel arrays. This would also make the length check unnecessary.

```
40 function exitStrategyMultiple(address[] calldata strategies, uint256  
    ↪ [] calldata minNavDelta) external;
```

CVF-152 INFO

- **Category** Bad datatype
- **Source** ISwapRouter.sol

Recommendation The type for these fields should be "IERC20".

```
9 address tokenIn; // Token to swap from
10 address tokenOut; // Token to receive
```

CVF-153 INFO

- **Category** Bad datatype
- **Source** ISwapRouter.sol

Recommendation The parameter type should be "ISwapAdapter".

```
16 event SwapAdapterWhitelisted(address indexed adapter);
   event SwapAdapterBlacklisted(address indexed adapter);

26 error AdapterNotWhitelisted(address adapter);
```

CVF-154 INFO

- **Category** Bad datatype
- **Source** ISwapRouter.sol

Recommendation The type for these parameters should be "IERC20".

```
20 address indexed fromToken,
   address indexed toToken,
```

CVF-155 INFO

- **Category** Bad datatype
- **Source** ICrossChainContainer.sol

Recommendation The type for this field should be more specific.

```
6 address adapter;

11 address tokenOnDestinationChain;
```

CVF-156 INFO

- **Category** Bad naming

- **Source** ICrossChainContainer.sol

Recommendation Events are usually named via nouns, such as "MessageRouter" or "TokenClaim".

```
18 event MessageRouterUpdated(address previousMessageRouter, address
    ↳ newMessageRouter);
event TokenClaimed(address token, uint256 amount);
20 event BridgeAdapterUpdated(address bridgeAdapter, bool isSupported);
event PeerContainerUpdated(address previousPeerContainer, address
    ↳ newPeerContainer);
event RemoteChainIdUpdated(uint256 previousRemoteChainId, uint256
    ↳ newRemoteChainId);
event BridgeSent(address token, uint256 amount, address
    ↳ bridgeAdapter, address bridgeTo);
```

CVF-157 INFO

- **Category** Suboptimal

- **Source** ICrossChainContainer.sol

Description The previous value parameters are redundant, as their values could be derived from the previous events.

Recommendation Remove the redundant parameters.

```
18 event MessageRouterUpdated(address previousMessageRouter, address
    ↳ newMessageRouter);

21 event PeerContainerUpdated(address previousPeerContainer, address
    ↳ newPeerContainer);
event RemoteChainIdUpdated(uint256 previousRemoteChainId, uint256
    ↳ newRemoteChainId);
```

CVF-158 INFO

- **Category** Procedural
- **Source** ICrossChainContainer.sol

Recommendation Address parameters should be indexed.

```
18 event MessageRouterUpdated(address previousMessageRouter, address
    ↳ newMessageRouter);
event TokenClaimed(address token, uint256 amount);
20 event BridgeAdapterUpdated(address bridgeAdapter, bool isSupported);
event PeerContainerUpdated(address previousPeerContainer, address
    ↳ newPeerContainer);

23 event BridgeSent(address token, uint256 amount, address
    ↳ bridgeAdapter, address bridgeTo);
```

CVF-159 INFO

- **Category** Bad datatype
- **Source** ICrossChainContainer.sol

Recommendation The type for parameters should be "IMessageRouter".

```
18 event MessageRouterUpdated(address previousMessageRouter, address
    ↳ newMessageRouter);
```

CVF-160 INFO

- **Category** Bad datatype
- **Source** ICrossChainContainer.sol

Recommendation The type for the "token" parameters should be "IERC20".

```
19 event TokenClaimed(address token, uint256 amount);

23 event BridgeSent(address token, uint256 amount, address
    ↳ bridgeAdapter, address bridgeTo);
```

CVF-161 INFO

- **Category** Bad datatype
- **Source** ICrossChainContainer.sol

Recommendation The type for the "bridgeAdapter" parameter should be "IBridgeAdapter".

```
20 event BridgeAdapterUpdated(address bridgeAdapter, bool isSupported);
```

CVF-162 INFO

- **Category** Bad datatype
- **Source** ICrossChainContainer.sol

Recommendation The type for parameters should be more specific.

```
21 event PeerContainerUpdated(address previousPeerContainer, address  
    ↪ newPeerContainer);
```

CVF-163 INFO

- **Category** Procedural
- **Source** ICrossChainContainer.sol

Recommendation These errors could be made more useful by adding certain parameters into them.

```
25 error TokenNotExpected();  
error UnclaimedTokens();  
  
28 error NotExpectingTokens();  
error BridgeAdapterNotSupported();  
30 error SameBridgeAdapterStatus();  
error PeerContainerAlreadySet();  
error RemoteChainIdAlreadySet();  
error InvalidDecimals();
```

CVF-164 INFO

- **Category** Bad datatype
- **Source** ICrossChainContainer.sol

Recommendation The argument type should be "IMessageRouter".

```
36 function setMessageRouter(address newMessageRouter) external;
```

CVF-165 INFO

- **Category** Bad datatype
- **Source** ICrossChainContainer.sol

Recommendation The argument type should be more specific.

```
38 function setPeerContainer(address newPeerContainer) external;
```

CVF-166 INFO

- **Category** Bad datatype
- **Source** ICrossChainContainer.sol

Recommendation The type for the "bridgeAdapter" argument should be more specific.

```
40 function setBridgeAdapter(address bridgeAdapter, bool isSupported)
    ↪ external;
```

CVF-167 INFO

- **Category** Bad datatype
- **Source** ICrossChainContainer.sol

Recommendation The return type should be more specific.

```
42 function peerContainer() external view returns (address);
```

CVF-168 INFO

- **Category** Bad datatype
- **Source** IContainerPrincipal.sol

Recommendation The type for this field should be "IERC20[]".

```
21 address[] tokens;
```

CVF-169 INFO

- **Category** Suboptimal
- **Source** IContainerPrincipal.sol

Recommendation It would be more efficient to use a single array of structs with two fields, rather than two parallel arrays.

```
21 address[] tokens;
    uint256[] minAmounts;
```

CVF-170 INFO

- **Category** Bad naming
- **Source** IContainerPrincipal.sol

Recommendation Events are usually named via nouns, such as “DepositRequest” or “WithdrawalRequest”.

```
31 event DepositRequestRegistered(uint256 amount);
event WithdrawalRequestRegistered(uint256 shares);
event DepositRequestSent();
event WithdrawalRequestSent(uint256 shares);
event DepositResponseReceived(uint256 claimCounter, uint256 nav0,
    ↪ uint256 nav1);
event WithdrawalResponseReceived(uint256 claimCounter);
event DepositReported(uint256 nav0, uint256 nav1, uint256 remainder)
    ↪ ;
event WithdrawalReported();
```

CVF-171 INFO

- **Category** Bad datatype
- **Source** IContainerPrincipal.sol

Recommendation The type for this argument should be “IBridgeAdapter[]”.

```
45 address[] calldata bridgeAdapters,
```

CVF-172 INFO

- **Category** Bad datatype
- **Source** IContainerPrincipal.sol

Recommendation The type for the “bridgeAdapter” argument should be “IBridgeAdapter”.

```
51 function claim(address bridgeAdapter, address token) external;
```

CVF-173 INFO

- **Category** Bad datatype
- **Source** IContainerPrincipal.sol

Recommendation The type for the “token” argument should be “IERC20”.

```
51 function claim(address bridgeAdapter, address token) external;
```

CVF-174 INFO

- **Category** Bad datatype

- **Source** IMessageAdapter.sol

Recommendation The return type should be "IMessageRouter".

```
6 function router() external view returns (address);
```

CVF-175 FIXED

- **Category** Unclear behavior

- **Source** IMessageAdapter.sol

Description This function should emit some event, and this event should be declared in this interface.

```
7 function send(uint256 chainTo, bytes memory params, bytes memory  
  ↳ rawMessage) external payable;
```

CVF-176 INFO

- **Category** Documentation

- **Source** IMessageReceiver.sol

Description The comment describes details of a particular implementation, while interfaces usually try to be implementation agnostic.

Recommendation Rephrase the comment to describe the contract of the function, rather than particular implementation details.

```
6 // Decode raw message using MessagingLib library  
function receiveMessage(bytes memory rawMessage) external;
```

CVF-177 INFO

- **Category** Bad datatype

- **Source** IOneInchAdapter.sol

Recommendation The parameter type should be "IERC20".

```
7 error InvalidSourceToken(address token);  
error InvalidDestinationToken(address token);
```

CVF-178 INFO

- **Category** Bad datatype
- **Source** IPriceOracle.sol

Recommendation The type for the "token" arguments should be "IERC20".

```
5 function getPrice(address token) external view returns (uint256
    ↳ price, uint256 decimals);
function getUsdValue(address token, uint256 amount) external view
    ↳ returns (uint256);
function getUsdValueUnified(address token, uint256 amount) external
    ↳ view returns (uint256);
```

CVF-179 INFO

- **Category** Documentation
- **Source** IPriceOracle.sol

Description The number format of the returned value is unclear.

Recommendation Explain in a documentation comment.

```
6 function getUsdValue(address token, uint256 amount) external view
    ↳ returns (uint256);
function getUsdValueUnified(address token, uint256 amount) external
    ↳ view returns (uint256);
```

CVF-180 INFO

- **Category** Bad datatype
- **Source** IStrategyContainer.sol

Recommendation The type for the "strategy" parameter should be more specific.

```
24 event StrategyAdded(address indexed strategy);
event StrategyRemoved(address indexed strategy);
event StrategyEntered(address indexed strategy, uint256 nav0,
    ↳ uint256 nav1, bool hasRemainder);
event StrategyExited(address indexed strategy, uint256 share);
event StrategyInputTokensUpdated(address indexed strategy);
```

```
38 error StrategyAlreadyHarvested(address strategy);
```

```
40 error StrategyAlreadyEntered(address strategy);
error StrategyAlreadyExited(address strategy);
error StrategyHarvestOutdated(address strategy);
```

CVF-181 INFO

- **Category** Bad naming

- **Source** IStrategyContainer.sol

Recommendation Events are usually named via nouns, such as "Strategy" or "StrategyRemoval".

```
24 event StrategyAdded(address indexed strategy);
event StrategyRemoved(address indexed strategy);
event StrategyEntered(address indexed strategy, uint256 nav0,
    ↳ uint256 nav1, bool hasRemainder);
event StrategyExited(address indexed strategy, uint256 share);
event StrategyInputTokensUpdated(address indexed strategy);
event TreasuryUpdated(address oldTreasury, address newTreasury);
30 event FeePctUpdated(uint256 oldFeePct, uint256 newFeePct);
event BridgeCollectorUpdated(address oldBridgeCollector, address
    ↳ newBridgeCollector);
event PriceOracleUpdated(address oldPriceOracle, address
    ↳ newPriceOracle);
event ReshufflingModeUpdated(bool reshufflingMode);
```

CVF-182 INFO

- **Category** Suboptimal

- **Source** IStrategyContainer.sol

Description The old value parameters are redundant, as their values could be derived from the previous events.

Recommendation Remove the redundant parameters.

```
29 event TreasuryUpdated(address oldTreasury, address newTreasury);
30 event FeePctUpdated(uint256 oldFeePct, uint256 newFeePct);
event BridgeCollectorUpdated(address oldBridgeCollector, address
    ↳ newBridgeCollector);
event PriceOracleUpdated(address oldPriceOracle, address
    ↳ newPriceOracle);
```

CVF-183 INFO

- **Category** Procedural

- **Source** IStrategyContainer.sol

Recommendation The parameters should be indexed.

```
29 event TreasuryUpdated(address oldTreasury, address newTreasury);
```

```
31 event BridgeCollectorUpdated(address oldBridgeCollector, address  
    ↪ newBridgeCollector);  
event PriceOracleUpdated(address oldPriceOracle, address  
    ↪ newPriceOracle);
```

CVF-184 INFO

- **Category** Bad datatype

- **Source** IStrategyContainer.sol

Recommendation The type for the parameters should be "IPriceOracle".

```
32 event PriceOracleUpdated(address oldPriceOracle, address  
    ↪ newPriceOracle);
```

CVF-185 INFO

- **Category** Procedural

- **Source** IStrategyContainer.sol

Recommendation These errors could be made more useful by adding certain parameters into them.

```
37 error StrategyNotFound();
```

```
39 error StrategyAlreadyExists();
```

```
43 error IncorrectNav0();
```

```
45 error ActionUnavailableNotInReshufflingMode();  
error ActionUnavailableInReshufflingMode();
```

CVF-186 INFO

- **Category** Bad datatype
- **Source** IStrategyContainer.sol

Recommendation The return type should be "IPriceOracle".

```
54 function priceOracle() external view returns (address);
```

CVF-187 INFO

- **Category** Bad datatype
- **Source** IReshufflingGateway.sol

Recommendation The type for the "token" parameters should be indexed.

```
5 event WithdrawInRepairingMode(address indexed owner, address indexed
  ↳ token, uint256 amount);
event SentToLocalContainer(address indexed container, address
  ↳ indexed token, uint256 amount);
event SentToCrossChainContainer(address indexed container, address
  ↳ indexed token, uint256 amount);

9 event TokenWhitelisted(address indexed token);
10 event TokenBlacklisted(address indexed token);
```

CVF-188 INFO

- **Category** Bad datatype
- **Source** IReshufflingGateway.sol

Recommendation The type for the "container" parameter should be "IContainer".

```
6 event SentToLocalContainer(address indexed container, address
  ↳ indexed token, uint256 amount);
event SentToCrossChainContainer(address indexed container, address
  ↳ indexed token, uint256 amount);
```

CVF-189 INFO

- **Category** Bad naming
- **Source** IReshufflingGateway.sol

Recommendation Events are usually named via nouns.

```
9 event TokenWhitelisted(address indexed token);
10 event TokenBlacklisted(address indexed token);
event BridgeAdapterWhitelisted(address indexed bridgeAdapter);
event BridgeAdapterBlacklisted(address indexed bridgeAdapter);
```

CVF-190 INFO

- **Category** Bad datatype
- **Source** IReshufflingGateway.sol

Recommendation The parameter type should be "IBridgeAdapter".

```
11 event BridgeAdapterWhitelisted(address indexed bridgeAdapter);  
event BridgeAdapterBlacklisted(address indexed bridgeAdapter);  
  
21 error NotWhitelistedBridgeAdapter(address bridgeAdapter);
```

CVF-191 INFO

- **Category** Bad datatype
- **Source** IReshufflingGateway.sol

Recommendation The parameter type should be "IContainer".

```
14 error NotContainer(address container);
```

CVF-192 INFO

- **Category** Bad datatype
- **Source** IReshufflingGateway.sol

Recommendation The parameter type should be "IVault".

```
15 error NotVault(address vault);
```

CVF-193 INFO

- **Category** Bad datatype
- **Source** IReshufflingGateway.sol

Recommendation The parameter type should be "IERC20".

```
20 error NotWhitelistedToken(address token);  
  
23 error TokenNotWhitelistedOnContainer(address token);
```

CVF-194 INFO

- **Category** Bad datatype
- **Source** IMessageRouter.sol

Recommendation The type for this field should be "IMessageAdapter".

```
7 address adapter;
```

CVF-195 INFO

- **Category** Bad naming
- **Source** IMessageRouter.sol

Recommendation Events are usually named via nouns.

```
20 event MessageSent(
```

```
27 event MessageReceived(uint256 nonce, bytes32 path, address indexed
    ↳ receiver, address indexed adapter);
event MessageRetried(uint256 nonce, uint256 chainTo, bytes32
    ↳ remotePath, address indexed adapter);
event MessageRemovedFromCache(uint256 nonce, uint256 chainId,
    ↳ bytes32 remotePath);
30 event AdapterWhitelisted(address adapter);
event AdapterBlacklisted(address adapter);
event PathWhitelisted(address sender, address receiver, uint256
    ↳ chainId, bytes32 path);
event PathBlacklisted(address sender, address receiver, uint256
    ↳ chainId, bytes32 path);
event ReceiverSet(address sender, address receiver, uint256 chainId,
    ↳ bytes32 path);
```

CVF-196 INFO

- **Category** Procedural
- **Source** IMessageRouter.sol

Recommendation This parameter should be indexed.

```
22 uint256 chainTo,
```

CVF-197 INFO

- **Category** Bad datatype
- **Source** IMessageRouter.sol

Recommendation The type for this parameter should be "IMessageAdapter".

25 `address adapter`

CVF-198 INFO

- **Category** Bad datatype
- **Source** IMessageRouter.sol

Recommendation The type for the "adapter" parameter should be "IMessageAdapter".

27 `event MessageReceived(uint256 nonce, bytes32 path, address indexed
↪ receiver, address indexed adapter);`

30 `event AdapterWhitelisted(address adapter);
event AdapterBlacklisted(address adapter);`

CVF-199 INFO

- **Category** Procedural
- **Source** IMessageRouter.sol

Recommendation The "chainTo" parameter should be indexed.

28 `event MessageRetried(uint256 nonce, uint256 chainTo, bytes32
↪ remotePath, address indexedadapter);`

CVF-200 INFO

- **Category** Procedural
- **Source** IMessageRouter.sol

Recommendation The type for the "indexedAdapter" parameter should be "IMessageAdapter".

28 `event MessageRetried(uint256 nonce, uint256 chainTo, bytes32
↪ remotePath, address indexedadapter);`

CVF-201 INFO

- **Category** Procedural

- **Source** IMessageRouter.sol

Recommendation The "chainIdParameter should be indexed".

```
29 event MessageRemovedFromCache(uint256 nonce, uint256 chainId,  
    ↳ bytes32 remotePath);  
  
32 event PathWhitelisted(address sender, address receiver, uint256  
    ↳ chainId, bytes32 path);  
event PathBlacklisted(address sender, address receiver, uint256  
    ↳ chainId, bytes32 path);  
event ReceiverSet(address sender, address receiver, uint256 chainId,  
    ↳ bytes32 path);
```

CVF-202 INFO

- **Category** Procedural

- **Source** IMessageRouter.sol

Recommendation The "sender" and "receiver" parameters should be indexed.

```
32 event PathWhitelisted(address sender, address receiver, uint256  
    ↳ chainId, bytes32 path);  
event PathBlacklisted(address sender, address receiver, uint256  
    ↳ chainId, bytes32 path);  
event ReceiverSet(address sender, address receiver, uint256 chainId,  
    ↳ bytes32 path);
```

CVF-203 INFO

- **Category** Bad datatype

- **Source** IMessageRouter.sol

Recommendation The parameter type should be "IMessageAdapter".

```
36 error UnsupportedAdapter(address);  
  
39 error AdapterNotWhitelisted(address);
```

CVF-204 INFO

- **Category** Bad datatype
- **Source** IMessageRouter.sol

Recommendation The argument type should be "IMessageAdapter".

```
48 function whitelistMessageAdapter(address adapter) external;
```

```
50 function blacklistMessageAdapter(address adapter) external;
```

CVF-205 INFO

- **Category** Suboptimal
- **Source** IStrategyTemplate.sol

Recommendation It would be more efficient to use a single array of structures with three fields, rather than three parallel arrays.

```
16 uint256[] amountsBeforeExit;  
   address[] outputTokens;  
   uint256[] remainingAmounts;
```

CVF-206 INFO

- **Category** Bad datatype
- **Source** IStrategyTemplate.sol

Recommendation The type for these fields should be "IERC20".

```
17 address[] outputTokens;
```

```
33 address[] inputTokens;
```

CVF-207 INFO

- **Category** Bad naming
- **Source** IStrategyTemplate.sol

Recommendation Events are usually named via nouns, such as "Enter" or "Exit".

```
36 event Entered(uint256 navBefore, uint256 navAfter, bool hasRemainder  
   ↪ );  
   event Exited(uint256 navBefore, uint256 navAfter, bytes32 stateId);  
   event EmergencyExited(bytes32 toStateId);  
   event Harvested(uint256 navAfter);
```

CVF-208 INFO

- **Category** Documentation
- **Source** IStrategyTemplate.sol

Description The semantics of the returned values is unclear.

Recommendation Give descriptive names to the returned values and/or explain in a documentation comment.

```
50 ) external payable returns (uint256, bool, uint256[] memory);  
  
54 function exit(uint256 share, uint256 minNavDelta) external payable  
    ↪ returns (address[] memory, uint256[] memory);  
  
56 function harvest() external payable returns (uint256);
```

CVF-209 INFO

- **Category** Bad datatype
- **Source** IStrategyTemplate.sol

Recommendation The return type should be "IERC20".

```
64 function inputTokens() external view returns (address[] memory);  
  
66 function outputTokens() external view returns (address[] memory);
```

CVF-210 INFO

- **Category** Bad datatype
- **Source** ISwapAdapter.sol

Recommendation The type for the "token" parameter should be "IERC20".

```
5 error SlippageNotMet(address token, uint256 amountOut, uint256  
    ↪ minAmountOut);
```

CVF-211 INFO

- **Category** Bad datatype
- **Source** ISwapAdapter.sol

Recommendation The type for these arguments should be "IERC20".

```
8 address tokenIn,  
  address tokenOut,
```

CVF-212 INFO

- **Category** Bad datatype
- **Source** IUniswapV3Adapter.sol

Recommendation The type for the token arguments should be "IERC20".

```
8 function whitelistPath(address tokenIn, address tokenOut, uint24 fee  
  ↪ ) external;  
function blacklistPath(address tokenIn, address tokenOut, uint24 fee  
  ↪ ) external;
```

CVF-213 INFO

- **Category** Bad datatype
- **Source** IUniversalAdapter.sol

Recommendation The type for the field should be "ISwapRouter".

```
6 address router;
```

CVF-214 INFO

- **Category** Bad datatype
- **Source** IUniversalAdapter.sol

Recommendation The type for the token parameters should be "IERC20".

```
11 event SwapInfoSet(address indexed fromToken, address indexed toToken  
  ↪ , SwapInfo swapInfo);
```

```
14 error NoSwapInfo(address fromToken, address toToken);
```

CVF-215 INFO

- **Category** Bad datatype
- **Source** IUniversalAdapter.sol

Recommendation The type for the arguments should be "IERC20".

```
16 function isSwapSupported(address tokenIn, address tokenOut) external  
  ↪ view returns (bool);
```

CVF-216 INFO

- **Category** Bad datatype
- **Source** IUniversalAdapter.sol

Recommendation The type for these arguments should be "IERC20".

18 `address fromToken,`
`address toToken,`

CVF-217 INFO

- **Category** Bad datatype
- **Source** IUniversalAdapter.sol

Recommendation The type for this argument should be "IMessageRouter".

20 `address router,`

CVF-218 INFO

- **Category** Procedural
- **Source** IVault.sol

Description This version requirement is inconsistent with other files in the same code base.

Recommendation Use consistent version requirements across code base.

2 `pragma solidity ^0.8.22;`

CVF-219 INFO

- **Category** Bad naming

- **Source** IVault.sol

Recommendation Events are usually named via nouns, such as "MaxDepositAmount" or "MinDepositAmount".

```
86 event MaxDepositAmountUpdated(uint256 maxDepositAmount);
event MinDepositAmountUpdated(uint256 minDepositAmount);
event MaxDepositBatchSizeUpdated(uint256 maxDepositBatchSize);
event MinDepositBatchSizeUpdated(uint256 minDepositBatchSize);
90 event MinWithdrawBatchRatioUpdated(uint256 minWithdrawBatchRatio);

92 event ContainerAdded(address container);
event ContainerRemoved(address container);
event ContainerWeightsUpdated(address[] containers, uint256[]
    ↳ weights);

97 event DepositClaimed(

106 event WithdrawClaimed(address indexed user, address indexed
    ↳ onBehalfOf, uint256 indexed batchId, uint256 amount);

108 event DepositBatchProcessingStarted(uint256 batchId, uint256
    ↳ batchAmount);
event DepositBatchSkipped(uint256 batchId, uint256 batchAmount);
110 event DepositReportReceived(

117 event DepositBatchProcessingFinished(

124 event WithdrawBatchProcessingStarted(uint256 batchId, uint256
    ↳ batchShares, uint256 batchSharesPercent);
event WithdrawBatchSkipped(uint256 batchId, uint256 batchAmount);
event WithdrawReportReceived(address indexed container, uint256
    ↳ indexed batchId, uint256 notionAmount);
event WithdrawBatchProcessingFinished(uint256 batchId, uint256
    ↳ withdrawnNotionAmount);

event ReshufflingGatewayUpdated(address indexed previousGateway,
    ↳ address indexed newGateway);
130 event ReshufflingModeUpdated(bool reshufflingMode);
event RepairingModeSet();
event ReshufflingGatewayClaimed(address account);
```

CVF-220 INFO

- **Category** Bad datatype
- **Source** IVault.sol

Recommendation The parameter type should be "IContainer".

```
92 event ContainerAdded(address container);  
event ContainerRemoved(address container);  
  
111 address indexed container,
```

CVF-221 INFO

- **Category** Bad datatype
- **Source** IVault.sol

Recommendation The parameter should be indexed.

```
92 event ContainerAdded(address container);  
event ContainerRemoved(address container);
```

CVF-222 INFO

- **Category** Bad datatype
- **Source** IVault.sol

Recommendation The type of the "containers" parameter . should be "IContainer[]".

```
94 event ContainerWeightsUpdated(address[] containers, uint256[]  
    ↪ weights);
```

CVF-223 INFO

- **Category** Bad datatype
- **Source** IVault.sol

Recommendation The type of the "container" parameter should be "iContainer".

```
126 event WithdrawReportReceived(address indexed container, uint256  
    ↪ indexed batchId, uint256 notionAmount);
```

CVF-224 INFO

- **Category** Procedural

- **Source** IVault.sol

Recommendation These errors could be made more useful by adding certain parameters into them.

```
143 error ContainerAlreadyReported();  
error ContainerNotReallocating();  
error NoContainers();
```

```
147 error NotEnoughNotion();  
error NotEnoughSharesWithdrawn();
```

```
155 error IncorrectBatchId();  
error IncorrectBatchStatus();  
error IncorrectReport();  
error IncorrectStatus();  
error IncorrectWeights(uint256 weightsSum);
```

```
162 error DepositBatchCapReached();  
error DepositBatchSizeTooSmall();  
error CannotSkipBatch();  
error IncorrectNotionDistribution();  
error MaxContainersReached();  
error MissingContainerReport();
```

CVF-225 INFO

- **Category** Bad datatype

- **Source** IVault.sol

Recommendation The argument type should be "IContainer".

```
179 function isContainer(address container) external view returns (bool)  
    ↪ ;
```

CVF-226 INFO

- **Category** Bad datatype

- **Source** IVault.sol

Recommendation The type for the first returned value should be "IContainer[]".

```
185 function getContainers() external view returns (address[] memory,  
    ↪ uint256[] memory);
```

CVF-227 INFO

- **Category** Suboptimal
- **Source** IVault.sol

Recommendation It would be more efficient to return a single array of structs with two fields, rather than two parallel arrays.

```
185 function getContainers() external view returns (address[] memory,  
    ↪ uint256[] memory);
```

CVF-228 INFO

- **Category** Bad datatype
- **Source** IBridgeAdapter.sol

Recommendation The type for this field should be "IERC20".

```
9 address token;
```

CVF-229 INFO

- **Category** Bad naming
- **Source** IBridgeAdapter.sol

Recommendation Events are usually named via nouns, such as "BridgeSending", "Bridge", or "Claim".

```
13 event BridgeSent(address indexed token, uint256 amount, uint256  
    ↪ chain);  
event Bridged(address indexed claimer, address indexed token,  
    ↪ uint256 amount);  
event Claimed(address indexed claimer, address indexed token,  
    ↪ uint256 amount);  
event BridgePathUpdated(address indexed src, uint256 indexed chain,  
    ↪ address indexed dst);  
event PeerUpdated(uint256 indexed chain, address indexed peer);  
event BridgerWhitelisted(address indexed bridger);  
event BridgerBlacklisted(address indexed bridger);
```

CVF-230 INFO

- **Category** Bad datatype

- **Source** IBridgeAdapter.sol

Recommendation The type for the "token" parameters should be "IERC20".

```
13 event BridgeSent(address indexed token, uint256 amount, uint256  
    ↪ chain);  
event Bridged(address indexed claimer, address indexed token,  
    ↪ uint256 amount);  
event Claimed(address indexed claimer, address indexed token,  
    ↪ uint256 amount);
```

CVF-231 INFO

- **Category** Bad datatype

- **Source** IBridgeAdapter.sol

Recommendation The type for the "srcToken" parameter should be "IERC20".

```
22 error BadBridgePath(address srcToken, uint256 chainTo);
```

CVF-232 INFO

- **Category** Documentation

- **Source** IBridgeAdapter.sol

Description The semantics of the returned value is unclear.

Recommendation Give a descriptive name to the returned value and/or explain in a documentation comment.

```
25 function bridge(BridgeInstruction calldata bridgeInstruction,  
    ↪ address receiver) external returns (uint256);
```

```
29 function bridgePaths(address token, uint256 chain) external view  
    ↪ returns (address);
```

CVF-233 INFO

- **Category** Bad datatype
- **Source** IBridgeAdapter.sol

Recommendation The type for the "token" arguments should be "IERC20".

```
27 function claim(address token) external;  
29 function bridgePaths(address token, uint256 chain) external view  
    ↪ returns (address);
```

CVF-234 INFO

- **Category** Bad naming
- **Source** IBridgeAdapter.sol

Description Despite the name, this function returns a single path.

Recommendation Rename to "bridgePath".

```
29 function bridgePaths(address token, uint256 chain) external view  
    ↪ returns (address);
```

CVF-235 INFO

- **Category** Procedural
- **Source** AcrossBridgeAdapter.sol

Recommendation Consider importing these interfaces directly from the Across repo.

```
6 import {IAcrossV3Receiver} from "../dependencies/interfaces/across-  
    ↪ v3/IAcrossV3Receiver.sol";  
import {ISpookyPool} from "../dependencies/interfaces/across-v3/  
    ↪ ISpookyPool.sol";
```

CVF-236 INFO

- **Category** Bad datatype
- **Source** AcrossBridgeAdapter.sol

Recommendation The type for this variable should be "ISpookyPool".

```
15 address public override spookyPool;
```

CVF-237 INFO

- **Category** Bad naming
- **Source** AcrossBridgeAdapter.sol

Description It seems that despite the name, bae points are used here not percents.

Recommendation Rename the variable and the constant to actually refer base points instead of percents.

```
16 uint256 public feeCapPct;  
uint256 public constant MAX_FEE_CAP_PCT = 10_000; // 100%
```

CVF-238 INFO

- **Category** Bad datatype
- **Source** AcrossBridgeAdapter.sol

Recommendation The type for this argument should be "ISpookyPool".

```
22 address _spookyPool,
```

CVF-239 INFO

- **Category** Bad datatype
- **Source** AcrossBridgeAdapter.sol

Recommendation The argument type should be "ISpookyPool".

```
30 function setSpookyPool(address spookyPoolAddress) external onlyRole(  
    ↪ GOVERNANCE_ROLE) {
```

CVF-240 INFO

- **Category** Bad datatype
- **Source** AcrossBridgeAdapter.sol

Recommendation The type for the "token" argument should be "IERC20".

```
38 function handleV3AcrossMessage(address token, uint256 amount,  
    ↪ address, bytes memory message) external {
```

CVF-241 FIXED

- **Category** Bad naming
- **Source** AcrossBridgeAdapter.sol

Description The error name doesn't seem relevant to the actual check performed.

Recommendation Rename the error or clearly explain how this check is related to the error.

```
77 require(  
    slippageDeltaPct < feeCapPct,  
    MinTokenAmountTooLow(instruction.amount, acrossParams.fee,  
80     ↪ instruction.minTokenAmount)  
);
```

CVF-242 FIXED

- **Category** Unclear behavior
- **Source** AcrossBridgeAdapter.sol

Description These events are emitted even if nothing actually changed.

```
87 emit SpookyPoolUpdated(oldPool, newSpookyPool);
```

```
94 emit FeeCapPctUpdated(oldFeeCapPct, newFeeCapPct);
```

CVF-243 INFO

- **Category** Bad naming
- **Source** BridgeAdapter.sol

Description The semantics of the keys and values in these mappings is unclear.

Recommendation Give descriptive names to the keys and/or explain in a documentation comment.

```
19 mapping(address => mapping(uint256 => address)) public bridgePaths;  
20 mapping(address => mapping(address => uint256)) public  
    ↪ claimableAmounts;  
mapping(address => bool) public whitelistedBridgers;  
mapping(uint256 => address) public peers;
```

CVF-244 INFO

- **Category** Bad datatype
- **Source** BridgeAdapter.sol

Recommendation The type for the first key should be "IERC20".

```
19 mapping(address => mapping(uint256 => address)) public bridgePaths;
```

CVF-245 INFO

- **Category** Bad datatype
- **Source** BridgeAdapter.sol

Recommendation The type for the second key should be "IERC20".

```
20 mapping(address => mapping(address => uint256)) public  
    ↪ claimableAmounts;
```

CVF-246 INFO

- **Category** Bad datatype
- **Source** BridgeAdapter.sol

Recommendation The type for the token arguments should be "IERC20".

```
36 function setBridgePath(address tokenOnSrc, uint256 chainId, address  
    ↪ tokenOnDst) external onlyRole(GOVERNANCE_ROLE) {
```

CVF-247 FIXED

- **Category** Unclear behavior
- **Source** BridgeAdapter.sol

Description These events are emitted even if nothing actually changed.

```
41 emit BridgePathUpdated(tokenOnSrc, chainId, tokenOnDst);
```

```
49 emit PeerUpdated(chainId, peer);
```

```
55 emit BridgerWhitelisted(bridger);
```

```
61 emit BridgerBlacklisted(bridger);
```

```
118 emit Bridged(claimer, token, amount);
```

CVF-248 INFO

- **Category** Bad naming
- **Source** BridgeAdapter.sol

Description The name is confusing, as this function actually just removes a bridger from the white list, rather than adds it into the black list. Actually, there is no such thing as black list.

Recommendation Rename this function.

```
58 function blacklistBridger(address bridger) external onlyRole(  
    ↪ GOVERNANCE_ROLE) {
```

CVF-249 INFO

- **Category** Documentation
- **Source** BridgeAdapter.sol

Description The semantics of the returned value is unclear.

Recommendation Give a descriptive name to the returned value and/or explain in a documentation comment.

```
67 ) external virtual override nonReentrant returns (uint256) {  
  
151 ) internal virtual returns (uint256);
```

CVF-250 INFO

- **Category** Suboptimal
- **Source** BridgeAdapter.sol

Description This check is redundant, as it is anyway possible to pass a dead receiver address.

Recommendation Remove this check.

```
68 require(receiver != address(0), Errors.ZeroAddress());
```

CVF-251 INFO

- **Category** Bad datatype
- **Source** BridgeAdapter.sol

Recommendation The argument type should be "IERC20".

```
82 function claim(address token) external virtual override nonReentrant  
    ↪ {
```

CVF-252 INFO

- **Category** Suboptimal

- **Source** BridgeAdapter.sol

Description Cache evictions are hard to predict. Relying on them is a bad practice.

Recommendation Use a more predictable mechanism for storing bridging operations that could be retried.

```
95 require(_cache.exists(key), RingCacheLibrary.DoesNotExists(key));
```

CVF-253 INFO

- **Category** Bad datatype

- **Source** BridgeAdapter.sol

Recommendation The type for the "token" argument should be "IERC20".

```
102 function _isCached(address token, uint256 chainTo, uint256 amount,  
    ↪ address receiver) private view returns (bool) {
```

```
112 function _finalizeBridge(address claimer, address token, uint256  
    ↪ amount) internal {
```

```
121 function _claim(address claimer, address token) internal {
```

CVF-254 INFO

- **Category** Suboptimal

- **Source** BridgeAdapter.sol

Description These checks are redundant, as it is anyway possible to pass a dead token or claimer address.

Recommendation Remove these checks.

```
114 require(claimer != address(0), Errors.ZeroAddress());  
    require(token != address(0), Errors.ZeroAddress());
```

```
122 require(claimer != address(0), Errors.ZeroAddress());  
    require(token != address(0), Errors.ZeroAddress());
```



CVF-255 INFO

- **Category** Suboptimal

- **Source** BridgeAdapter.sol

Description These checks are redundant, as it is anyway possible to pass a dead token or chain ID. The real validation is preformed later by checking the "bridgePaths" array.

Recommendation Remove these checks.

```
138 require(instruction.token != address(0), Errors.ZeroAddress());  
    require(instruction.chainTo > 0, Errors.ZeroAmount());
```

CVF-256 INFO

- **Category** Procedural

- **Source** BridgeAdapter.sol

Recommendation It is a good practice to put a comment into an empty block to explain why the block is empty.

```
153 function _validatePayload(bytes memory payload) internal virtual {}
```

CVF-257 FIXED

- **Category** Readability

- **Source** StrategyTemplate.sol

Recommendation Consider placing external imports together.

```
4 import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol  
   ↪ ";  
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/  
   ↪ SafeERC20.sol";  
import {EnumerableSet} from "@openzeppelin/contracts/utils/structs/  
   ↪ EnumerableSet.sol";  
import {Initializable} from "@openzeppelin/contracts/proxy/utils/  
   ↪ Initializable.sol";  
import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";  
  
14 import {AccessControlUpgradeable} from "@openzeppelin/contracts-  
   ↪ upgradeable/access/AccessControlUpgradeable.sol";
```

CVF-258 FIXED

- **Category** Bad datatype

- **Source** StrategyTemplate.sol

Recommendation The type for this variable should be "IContainerAgent".

```
22 address private _containerAgent;
```

CVF-259 FIXED

- **Category** Bad datatype

- **Source** StrategyTemplate.sol

Recommendation The type for these variables should be "IERC20[]".

```
26 address[] private _inputTokens;  
address[] private _outputTokens;
```

CVF-260 FIXED

- **Category** Bad datatype

- **Source** StrategyTemplate.sol

Recommendation The return type should be "IERC20".

```
51 function inputTokens() external view returns (address[] memory) {
```

```
55 function outputTokens() external view returns (address[] memory) {
```

CVF-261 FIXED

- **Category** Bad datatype

- **Source** StrategyTemplate.sol

Recommendation The argument type should be "IContainerAgent".

```
59 function initialize(address containerAgent) public virtual;
```

CVF-262 FIXED

- **Category** Bad datatype

- **Source** StrategyTemplate.sol

Recommendation The type for these arguments should be "IERC20[]".

```
62 address[] memory __inputTokens,  
address[] memory __outputTokens,
```

CVF-263 FIXED

- **Category** Bad datatype
- **Source** StrategyTemplate.sol

Recommendation The type for this argument should be "IContainerAgent".

```
64 address containerAgent
```

CVF-264 FIXED

- **Category** Suboptimal
- **Source** StrategyTemplate.sol

Description This check is redundant, as it is anyway possible to pass a dead container agent address.

Recommendation Remove this check.

```
66 require(containerAgent != address(0), Errors.ZeroAddress());
```

CVF-265 FIXED

- **Category** Suboptimal
- **Source** StrategyTemplate.sol

Description These checks are redundant, as it is anyway possible to pass a dead token address.

```
73 require(__inputTokens[i] != address(0), Errors.ZeroAddress());
```

```
77 require(__outputTokens[i] != address(0), Errors.ZeroAddress());
```

CVF-266 FIXED

- **Category** Unclear behavior
- **Source** StrategyTemplate.sol

Description This function should emit some event.

```
84 function _setOnlyTokenState(bytes32 stateId) internal {
```



CVF-267 FIXED

- **Category** Documentation

- **Source** StrategyTemplate.sol

Description The semantics of the returned values is unclear.

Recommendation Give descriptive names to the returned values and/or explain in a documentation comment.

```
91  ) public payable override notInEmergencyMode onlyContainerAgent
    ↪ returns (uint256, bool, uint256[] memory) {

134 ) public payable override onlyContainerAgent returns (address[]
    ↪ memory, uint256[] memory) {

275 ) private returns (uint256[] memory, bool) {

296 function _prepareFundsAfterEnter(address[] memory tokens) private
    ↪ returns (uint256[] memory, bool) {
```

CVF-268 FIXED

- **Category** Bad datatype

- **Source** StrategyTemplate.sol

Recommendation The type for the "token" argument should be "IERC20".

```
199 function _getUsdValue(address token, uint256 amount) internal view
    ↪ returns (uint256) {
```

CVF-269 FIXED

- **Category** Bad datatype

- **Source** StrategyTemplate.sol

Recommendation The return type should be "IPriceRouter".

```
204 function _getPriceOracle() internal view returns (address) {
```

CVF-270 FIXED

- **Category** Bad datatype

- **Source** StrategyTemplate.sol

Recommendation The return type should be "ISwapRouter".

```
210 function _getSwapRouter() internal view returns (address) {
```

CVF-271 FIXED

- **Category** Suboptimal

- **Source** StrategyTemplate.sol

Recommendation It would be more efficient to pass a single array of structs with two fields, rather than two parallel arrays.

```
272 address[] memory tokens,  
    uint256[] memory amountsBeforeExit,
```

CVF-272 FIXED

- **Category** Bad datatype

- **Source** StrategyTemplate.sol

Recommendation The type for the "tokens" argument should be "IERC20[]".

```
272 address[] memory tokens,
```

```
296 function _prepareFundsAfterEnter(address[] memory tokens) private  
    ↪ returns (uint256[] memory, bool) {
```

CVF-273 INFO

- **Category** Documentation

- **Source** Collector.sol

Description The semantics of the keys in this mapping is unclear.

Recommendation Give descriptive names to the keys and/or explain in a documentation comment.

```
16 mapping(uint256 => mapping(uint256 => bool)) public  
    ↪ positionClaimedBatch;
```

CVF-274 INFO

- **Category** Bad naming

- **Source** Collector.sol

Recommendation Events are usually named via noun, such as "Claim".

```
20 event Claimed(address user, uint256 positionId, uint256 amount);
```

CVF-275 INFO

- **Category** Procedural
- **Source** Collector.sol

Recommendation The “user” and “positionId” parameters should be indexed.

```
20 event Claimed(address user, uint256 positionId, uint256 amount);
```

CVF-276 INFO

- **Category** Procedural
- **Source** Collector.sol

Recommendation This error could be made more useful by adding certain parameters into it.

```
24 error AlreadyClaimed();
```

CVF-277 INFO

- **Category** Bad datatype
- **Source** Collector.sol

Recommendation The argument type should be “IVault”.

```
31 function initialize(address vault) public initializer {
```

CVF-278 INFO

- **Category** Procedural
- **Source** Container.sol

Description Declaring a top-level struct in a file named after a contract makes it harder navigating through code.

Recommendation Move the struct declaration into the contract or move it into a separate file.

```
16 struct ContainerInitParams {
```

CVF-279 INFO

- **Category** Bad datatype
- **Source** Container.sol

Recommendation The type for this field should be “IVault”.

```
17 address vault;
```

CVF-280 INFO

- **Category** Bad datatype
- **Source** Container.sol

Recommendation The type for this field should be "IERC20".

18 `address notion;`

CVF-281 INFO

- **Category** Bad datatype
- **Source** Container.sol

Recommendation The type for this field should be "ISwapRouter".

21 `address swapRouter;`

CVF-282 INFO

- **Category** Bad datatype
- **Source** Container.sol

Recommendation The type for this variable should be "IVault".

31 `address public vault;`

CVF-283 INFO

- **Category** Bad datatype
- **Source** Container.sol

Recommendation The type for this variable should be "IERC20".

32 `address public notion;`

CVF-284 INFO

- **Category** Bad datatype
- **Source** Container.sol

Recommendation The type for this variable should be "ISwapRouter".

34 `address public override swapRouter;`

CVF-285 INFO

- **Category** Bad datatype
- **Source** Container.sol

Recommendation The key type for this mapping should be "IERC20".

```
40 mapping(address => uint256) public _whitelistedTokensDustThresholds;
```

CVF-286 INFO

- **Category** Suboptimal
- **Source** Container.sol

Description This check is redundant, as it is anyway possible to pass a dead vault address.

Recommendation Remove this check.

```
48 require(params.vault != address(0), Errors.ZeroAddress());
```

CVF-287 INFO

- **Category** Bad datatype
- **Source** Container.sol

Recommendation The argument type should be "IERC20".

```
64 function whitelistToken(address token) external onlyRole(  
    ↪ TOKEN_MANAGER_ROLE) {
```

```
71 function blacklistToken(address token) external onlyRole(  
    ↪ TOKEN_MANAGER_ROLE) {
```

CVF-288 INFO

- **Category** Suboptimal
- **Source** Container.sol

Description This check is redundant, as it is anyway possible to pass a dead token address.

Recommendation Remove this check.

```
65 require(token != address(0), Errors.ZeroAddress());
```

CVF-289 INFO

- **Category** Bad naming

- **Source** Container.sol

Description The name is confusing, as this function actually removes the token from the while list, rather than adds it into the black list. There is no such thing as the black list in the contract.

Recommendation Rename the function.

```
71 function blacklistToken(address token) external onlyRole(  
    ↪ TOKEN_MANAGER_ROLE) {
```

CVF-290 INFO

- **Category** Bad datatype

- **Source** Container.sol

Recommendation The type for the "token" argument should be "IERC20".

```
78 function setWhitelistedTokenDustThreshold(address token, uint256  
    ↪ threshold) external onlyRole(TOKEN_MANAGER_ROLE) {
```

```
97 function isTokenWhitelisted(address token) external view override  
    ↪ returns (bool) {
```

```
101 function _isTokenWhitelisted(address token) internal view returns (  
    ↪ bool) {
```

```
105 function _validateToken(address token) internal view {
```

CVF-291 INFO

- **Category** Bad datatype

- **Source** Container.sol

Recommendation The argument type should be "ISwapRouter".

```
87 function setSwapRouter(address newSwapRouter) external onlyRole(  
    ↪ TOKEN_MANAGER_ROLE) {
```

```
128 function _setSwapRouter(address newSwapRouter) internal {
```

CVF-292 INFO

- **Category** Suboptimal

- **Source** Container.sol

Description This check is redundant, as it is anyway possible to pass a dead swap router address.

Recommendation Remove this check.

```
88 require(newSwapRouter != address(0), Errors.ZeroAddress());
```

CVF-293 INFO

- **Category** Suboptimal

- **Source** Container.sol

Description This check is redundant, as it is superseded by the already performed while list check.

```
107 require(token != address(0), Errors.ZeroAddress());
```

CVF-294 INFO

- **Category** Readability

- **Source** StrategyContainer.sol

Recommendation Consider placing external imports together.

```
4 import {EnumerableSet} from "@openzeppelin/contracts/utils/structs/  
  ↳ EnumerableSet.sol";  
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol  
  ↳ ";  
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/  
  ↳ SafeERC20.sol";
```

```
11 import {ReentrancyGuardUpgradeable} from "@openzeppelin/contracts-  
  ↳ upgradeable/utils/ReentrancyGuardUpgradeable.sol";  
import {Initializable} from "@openzeppelin/contracts-upgradeable/  
  ↳ proxy/utils/Initializable.sol";
```

CVF-295 INFO

- **Category** Bad datatype
- **Source** StrategyContainer.sol

Recommendation The key type for these mappings should be more specific.

```
29 mapping(address => uint256) internal _strategyNav0;  
30 mapping(address => uint256) internal _strategyNav1;
```

CVF-296 INFO

- **Category** Bad datatype
- **Source** StrategyContainer.sol

Recommendation The type for this variable should be "IPriceOracle".

```
38 address public override priceOracle;
```

CVF-297 INFO

- **Category** Bad datatype
- **Source** StrategyContainer.sol

Recommendation The argument type should be "IPriceOracle".

```
60 function setPriceOracle(address newPriceOracle) external onlyRole(  
    ↳ STRATEGY_MANAGER_ROLE) {
```

```
64 function _setPriceOracle(address newPriceOracle) internal {
```

CVF-298 INFO

- **Category** Suboptimal

- **Source** StrategyContainer.sol

Description These checks are redundant, as it is anyway possible to pass a dead address.

Recommendation Remove these checks.

```
65 require(newPriceOracle != address(0), Errors.ZeroAddress());
72 require(newTreasury != address(0), Errors.ZeroAddress());
86 require(newBridgeCollector != address(0), Errors.ZeroAddress());
115 require(strategy != address(0), Errors.ZeroAddress());
130 require(strategy != address(0), Errors.ZeroAddress());
```

CVF-299 INFO

- **Category** Unclear behavior

- **Source** StrategyContainer.sol

Description These events are emitted even if nothing actually changed.

```
68 emit PriceOracleUpdated(oldPriceOracle, priceOracle);
75 emit TreasuryUpdated(oldTreasury, treasury);
82 emit FeePctUpdated(previousFeePct, feePct);
89 emit BridgeCollectorUpdated(oldBridgeCollector, newBridgeCollector);
96 emit ReshufflingModeUpdated(true);
101 emit ReshufflingModeUpdated(false);
```

CVF-300 INFO

- **Category** Bad datatype
- **Source** StrategyContainer.sol

Recommendation The type for the “strategy” arguments should be more specific.

```
110 function isStrategy(address strategy) external view returns (bool) {
114 function addStrategy(address strategy) external onlyRole(
    ↳ STRATEGY_MANAGER_ROLE) {
129 function removeStrategy(address strategy) external onlyRole(
    ↳ STRATEGY_MANAGER_ROLE) {
143 function _isStrategy(address strategy) private view returns (bool) {
187 function _enterStrategy(address strategy, uint256[] calldata
    ↳ inputAmounts, uint256 minNavDelta) internal {
227     address strategy,
235     address strategy,
265     address strategy,
276 function _exitStrategyInReshufflingMode(address strategy, uint256
    ↳ share, uint256 minNavDelta) internal {
295 function _exitStrategy(address strategy, uint256 share, uint256
    ↳ minNavDelta) internal {
```

CVF-301 INFO

- **Category** Procedural
- **Source** CrossChainContainer.sol

Description Declaring a top-level structure in a file named after a contract makes it harder navigating through code.

Recommendation Move the struct declaration into the contract or move it into a separate file.

```
17 struct CrossChainContainerInitParams {
```

CVF-302 INFO

- **Category** Bad datatype
- **Source** CrossChainContainer.sol

Recommendation The type for this field should be "IMessageRouter".

```
18 address messageRouter;
```

CVF-303 INFO

- **Category** Bad datatype
- **Source** CrossChainContainer.sol

Recommendation The type for this field should be more specific.

```
20 uint256 remoteChainId;
```

CVF-304 INFO

- **Category** Bad datatype
- **Source** CrossChainContainer.sol

Recommendation The type for this variable should be "IMessageRouter".

```
29 address public messageRouter;
```

CVF-305 INFO

- **Category** Bad datatype
- **Source** CrossChainContainer.sol

Recommendation The type for this variable should be more specific.

```
30 address public override peerContainer;
```

CVF-306 INFO

- **Category** Documentation
- **Source** CrossChainContainer.sol

Description The semantics of this variable is unclear.

Recommendation Explain in a documentation comment.

```
33 uint256 public claimCounter;
```

CVF-307 INFO

- **Category** Documentation
- **Source** CrossChainContainer.sol

Description The semantics of the keys for these mappings is unclear.

Recommendation Give descriptive names to the keys and/or explain in a documentation comment.

```
34 mapping(address => uint256) private _expectedTokenAmounts;  
   mapping(address => bool) private _isBridgeAdapterSupported;
```

CVF-308 INFO

- **Category** Readability
- **Source** CrossChainContainer.sol

Recommendation This value could be rendered as "0.9e4".

```
37 uint256 public constant MAX_BRIDGE_SLIPPAGE = 9000; // 10%
```

CVF-309 INFO

- **Category** Bad datatype
- **Source** CrossChainContainer.sol

Recommendation The argument type should be "IMessageRouter".

```
57 function setMessageRouter(address newMessageRouter) external  
   ↳ onlyRole(MESSENGER_MANAGER_ROLE) {
```

```
71 function _setMessageRouter(address newMessageRouter) internal {
```

CVF-310 INFO

- **Category** Bad datatype
- **Source** CrossChainContainer.sol

Recommendation The argument type should be more specific.

```
61 function setPeerContainer(address newPeerContainer) external  
   ↳ onlyRole(MESSENGER_MANAGER_ROLE) {
```

```
84 function _setPeerContainer(address newPeerContainer) internal {
```

CVF-311 INFO

- **Category** Unclear behavior
- **Source** CrossChainContainer.sol

Description These events are emitted even if nothing actually changed.

```
75 emit MessageRouterUpdated(previousMessageRouter, newMessageRouter);  
81 emit RemoteChainIdUpdated(previousRemoteChainId, newRemoteChainId);  
87 emit PeerContainerUpdated(previousPeerContainer, newPeerContainer);
```

CVF-312 INFO

- **Category** Bad datatype
- **Source** CrossChainContainer.sol

Recommendation The type for the "tokens" argument should be "IERC20[]".

```
90 function _processExpectedTokens(address[] memory tokens, uint256[]  
    ↪ memory amounts) internal {
```

CVF-313 INFO

- **Category** Suboptimal
- **Source** CrossChainContainer.sol

Recommendation It would be more efficient otherwise pass a single array of structs with two fields, rather than two parallel arrays. This would also make the length check unnecessary.

```
90 function _processExpectedTokens(address[] memory tokens, uint256[]  
    ↪ memory amounts) internal {
```

CVF-314 INFO

- **Category** Unclear behavior
- **Source** CrossChainContainer.sol

Description This logic is unclear and the comment doesn't help a lot.

Recommendation Add more details and some context to the comment.

```
99 uint256 expectedAmount = _expectedTokenAmounts[tokens[i]];  
100 if (expectedAmount == 0) {  
    claimCounterCached += 1;  
}  
// NOTE: += in case same token is bridged via different adapters
```

CVF-315 INFO

- **Category** Bad datatype

- **Source** CrossChainContainer.sol

Recommendation The type for the "bridgeAdapter" argument should be "IBridgeAdapter".

```
113 function setBridgeAdapter(address bridgeAdapter, bool isSupported)
    ↪ external onlyRole(BRIDGE_ADAPTER_MANAGER_ROLE) {
```

```
121 function _claimExpectedToken(address bridgeAdapter, address token)
    ↪ internal {
```

```
142     address bridgeAdapter,
```

```
174     address bridgeAdapter,
```

```
182 function _validateBridgeAdapter(address bridgeAdapter) internal view
    ↪ {
```

```
192 function _approveTokenToBridgeAdapter(address token, address
    ↪ bridgeAdapter, uint256 amount) internal {
```

CVF-316 INFO

- **Category** Suboptimal

- **Source** CrossChainContainer.sol

Description This check is redundant, as it is anyway possible to pass a dead bridge adapter.

Recommendation Remove this check.

```
114 require(bridgeAdapter != address(0), Errors.ZeroAddress());
```

```
183 require(bridgeAdapter != address(0), Errors.ZeroAddress());
```

CVF-317 INFO

- **Category** Bad datatype
- **Source** CrossChainContainer.sol

Recommendation The type for the "token" argument should be "IERC20".

```
121 function _claimExpectedToken(address bridgeAdapter, address token)  
    ↪ internal {
```

```
187 function _validateClaimableToken(address token) internal view {
```

```
192 function _approveTokenToBridgeAdapter(address token, address  
    ↪ bridgeAdapter, uint256 amount) internal {
```

CVF-318 INFO

- **Category** Documentation
- **Source** CrossChainContainer.sol

Description The semantics of the returned values is unclear.

Recommendation Give descriptive names to the returned values and/or explain in a documentation comment.

```
145 ) internal returns (address, uint256) {
```

CVF-319 INFO

- **Category** Bad datatype
- **Source** CrossChainContainer.sol

Recommendation The type for this argument should be "IERC20".

```
175 address tokenOnSourceChain
```

CVF-320 INFO

- **Category** Suboptimal
- **Source** CrossChainContainer.sol

Description This check is redundant, as it is anyway possible to pass a dead token address.

Recommendation Remove this check.

```
194 require(token != address(0), Errors.ZeroAddress());
```

CVF-321 INFO

- **Category** Bad datatype
- **Source** ContainerAgent.sol

Recommendation The type for this argument should be more specific.

```
44 address strategy,
```

```
59 address[] calldata strategies,
```

CVF-322 INFO

- **Category** Suboptimal
- **Source** ContainerAgent.sol

Recommendation It would be more efficient to pass a single array of structs with three fields, rather than three parallel arrays. This would also make the length check unnecessary.

```
59 address[] calldata strategies,  
60 uint256[][] calldata inputAmounts,  
    uint256[] calldata minNavDelta
```

CVF-323 INFO

- **Category** Bad datatype
- **Source** ContainerAgent.sol

Recommendation The type for the strategy argument should be more specific.

```
79 function exitStrategy(address strategy, uint256 minNavDelta)  
    ↪ external onlyRole(OPERATOR_ROLE) {
```

CVF-324 INFO

- **Category** Bad datatype
- **Source** ContainerAgent.sol

Recommendation The type for this argument should be more specific.

```
93 address[] calldata strategies,
```

CVF-325 INFO

- **Category** Suboptimal
- **Source** ContainerAgent.sol

Recommendation It would be more efficient to pass a single array of structs with two fields rather than two parallel arrays. This would also make the length check unnecessary.

```
93  address[] calldata strategies,  
    uint256[] calldata minNavDeltas
```

CVF-326 INFO

- **Category** Bad datatype
- **Source** ContainerAgent.sol

Recommendation The type for this argument should be "IBridgeAdapter[]".

```
116  address[] calldata bridgeAdapters,
```

```
166  address[] calldata bridgeAdapters,
```

CVF-327 INFO

- **Category** Suboptimal
- **Source** ContainerAgent.sol

Recommendation It would be more efficient to pass a single array of structs with two fields, rather than two parallel arrays. This would also make the length check unnecessary.

```
116  address[] calldata bridgeAdapters,  
     IBridgeAdapter.BridgeInstruction[] calldata bridgeInstructions
```

```
166  address[] calldata bridgeAdapters,  
     IBridgeAdapter.BridgeInstruction[] calldata bridgeInstructions
```

CVF-328 INFO

- **Category** Suboptimal
- **Source** ContainerAgent.sol

Description This check is redundant, as it is anyway possible to pass a dead adapter address.

Recommendation Remove this check.

```
124  require(messageInstruction.adapter != address(0), Errors.ZeroAddress  
    ↪    ());
```

CVF-329 INFO

- **Category** Readability
- **Source** ContainerAgent.sol

Recommendation Should be "else if".

```
224 if (message.type_ == ContainerMessagePackingLib.  
    ↪ WITHDRAWAL_REQUEST_TYPE) {
```

CVF-330 INFO

- **Category** Readability
- **Source** ContainerAgent.sol

Recommendation Should be "else revert".

```
234 revert ContainerMessagePackingLib.WrongMessageType(message.type_);
```

CVF-331 INFO

- **Category** Bad datatype
- **Source** ContainerAgent.sol

Recommendation The type for the "bridgeAdapter" argument should be "IBridgeAdapter".

```
239 function claim(address bridgeAdapter, address token) external  
    ↪ onlyRole(OPERATOR_ROLE) nonReentrant {
```

```
249     address bridgeAdapter,
```

CVF-332 INFO

- **Category** Bad datatype
- **Source** ContainerAgent.sol

Recommendation The type for the "token" argument should be "IERC20".

```
239 function claim(address bridgeAdapter, address token) external  
    ↪ onlyRole(OPERATOR_ROLE) nonReentrant {
```

```
250     address token
```

CVF-333 INFO

- **Category** Bad datatype
- **Source** ContainerAgent.sol

Recommendation The type for the "bridgeAdapters" argument should be "IBridgeAdapter[]".

```
257 address[] calldata bridgeAdapters,
```

```
275 address[] memory bridgeAdapters,
```

CVF-334 INFO

- **Category** Bad datatype
- **Source** ContainerAgent.sol

Recommendation The type for the "tokens" argument should be "IERC20[]".

```
258 address[] calldata tokens
```

CVF-335 INFO

- **Category** Suboptimal
- **Source** ContainerAgent.sol

Recommendation It would be more efficient to pass a single array of structs with two fields, rather than two parallel arrays. This would also make the length check unnecessary.

```
257 address[] calldata bridgeAdapters,  
address[] calldata tokens
```

```
275 address[] memory bridgeAdapters,  
IBridgeAdapter.BridgeInstruction[] calldata instructions
```

CVF-336 INFO

- **Category** Unclear behavior
- **Source** ContainerLocal.sol

Description These functions should emit some events.

```
35 function registerDepositRequest(uint256 amount) external onlyVault {
```

```
42 function registerWithdrawRequest(uint256 amount) external onlyVault  
    ↪ {
```

CVF-337 INFO

- **Category** Bad datatype
- **Source** ContainerLocal.sol

Recommendation The type for this argument should be more specific.

```
78 address strategy,  
92 address[] calldata strategies,  
124 address[] calldata strategies,
```

CVF-338 INFO

- **Category** Suboptimal
- **Source** ContainerLocal.sol

Recommendation It would be more efficient to pass a single array of structs with three fields rather than three parallel arrays. This would also make the length check unnecessary.

```
92 address[] calldata strategies,  
uint256[][] calldata inputAmounts,  
uint256[] calldata minNavDelta
```

CVF-339 INFO

- **Category** Bad datatype
- **Source** ContainerLocal.sol

Recommendation The type for the "strategy" argument should be more specific.

```
111 function exitStrategy(address strategy, uint256 minNavDelta)  
    ↪ external onlyRole(OPERATOR_ROLE) {
```

CVF-340 INFO

- **Category** Suboptimal
- **Source** ContainerLocal.sol

Recommendation It would be more efficient to pass a single array of structs with two fields, rather than two parallel arrays. This would also make the length check unnecessary.

```
124 address[] calldata strategies,  
uint256[] calldata minNavDeltas
```

CVF-341 INFO

- **Category** Bad datatype
- **Source** ContainerPrincipal.sol

Recommendation The type for this argument should be "IBridgeAdapter[]".

```
68 address[] calldata bridgeAdapters,
```

```
224 address[] calldata bridgeAdapters,
```

CVF-342 INFO

- **Category** Suboptimal
- **Source** ContainerPrincipal.sol

Recommendation It would be more efficient to pass a single array of structs with two fields, rather than two parallel arrays. This would also make the length check unnecessary.

```
68 address[] calldata bridgeAdapters,  
    IBridgeAdapter.BridgeInstruction[] calldata bridgeInstructions
```

CVF-343 INFO

- **Category** Suboptimal
- **Source** ContainerPrincipal.sol

Description This check is redundant, as it is anyway possible to pass a dead adapter address.

Recommendation Remove this check.

```
74 require(messageInstruction.adapter != address(0), Errors.ZeroAddress  
    ↪ ());
```

CVF-344 INFO

- **Category** Bad datatype
- **Source** ContainerPrincipal.sol

Recommendation The type for the "bridgeAdapter" argument should be "IBridgeAdapter".

```
209 function claim(address bridgeAdapter, address token) external  
    ↪ onlyRole(OPERATOR_ROLE) nonReentrant {
```

CVF-345 INFO

- **Category** Bad datatype
- **Source** ContainerPrincipal.sol

Recommendation The type for the "token" argument should be "IERC20".

```
209 function claim(address bridgeAdapter, address token) external  
    ↪ onlyRole(OPERATOR_ROLE) nonReentrant {
```

CVF-346 INFO

- **Category** Bad datatype
- **Source** ContainerPrincipal.sol

Recommendation The type for this argument should be "IERC20[]".

```
225 address[] calldata tokens
```

CVF-347 INFO

- **Category** Suboptimal
- **Source** ContainerPrincipal.sol

Recommendation It would be more efficient otherwise pass a single array of structs with two fields, rather than two parallel arrays. This would also make the length check unnecessary.

```
224 address[] calldata bridgeAdapters,  
    address[] calldata tokens
```

CVF-348 INFO

- **Category** Bad datatype
- **Source** MessageRouter.sol

Recommendation The key type for this mapping should be "IMessageAdapter".

```
24 mapping(address => bool) private _whitelistedMessageAdapters;
```

CVF-349 INFO

- **Category** Bad naming
- **Source** MessageRouter.sol

Description The semantics of the keys for these mappings is unclear.

Recommendation Explain in a documentation comment.

```
25 mapping(bytes32 => bool) private _whitelistedPaths;  
   mapping(bytes32 => address) private _receivers;  
   mapping(bytes32 => uint256) private _lastNonces;
```

CVF-350 INFO

- **Category** Suboptimal
- **Source** MessageRouter.sol

Recommendation It would be more efficient to merge these mappings into a single mapping whose keys are path hashes and values are structs encapsulating the values of the original mappings.

```
25 mapping(bytes32 => bool) private _whitelistedPaths;  
   mapping(bytes32 => address) private _receivers;  
   mapping(bytes32 => uint256) private _lastNonces;
```

CVF-351 INFO

- **Category** Suboptimal
- **Source** MessageRouter.sol

Description These checks are redundant, as it is anyway possible to pass a dead sender, receiver, or chain ID.

Recommendation Remove these checks.

```
48 require(sender != address(0), Errors.ZeroAddress());  
   require(receiver != address(0), Errors.ZeroAddress());  
50 require(chainId > 0, Errors.ZeroAmount());
```

```
63 require(sender != address(0), Errors.ZeroAddress());  
   require(receiver != address(0), Errors.ZeroAddress());  
   require(chainId > 0, Errors.ZeroAmount());
```

```
78 require(sender != address(0), Errors.ZeroAddress());  
   require(chainId > 0, Errors.ZeroAmount());
```

CVF-352 INFO

- **Category** Procedural
- **Source** MessageRouter.sol

Recommendation Consider extractive this common logic into a utility function.

```
52 bytes32 path = keccak256(abi.encodePacked(sender, receiver, chainId)
    ↪ );
```

```
67 bytes32 path = keccak256(abi.encodePacked(sender, receiver, chainId)
    ↪ );
```

CVF-353 INFO

- **Category** Suboptimal
- **Source** MessageRouter.sol

Description These checks are redundant, as they are superseded by the white list check below.

Recommendation Remove these checks.

```
112 require(receiver != address(0), Errors.ZeroAddress());
    require(sendParams.chainTo > 0, Errors.ZeroAmount());
```

CVF-354 INFO

- **Category** Suboptimal
- **Source** MessageRouter.sol

Description The value "_receivers[path]" is read from the storage twice.

Recommendation Read once and reuse.

```
146 require(_receivers[path] != address(0), ReceiverNotSet(path));
```

```
149 IMessageReceiver(_receivers[path]).receiveMessage(rawMessage);
```

CVF-355 INFO

- **Category** Procedural
- **Source** PriceOracle.sol

Recommendation This interface should be moved into a separate file named "IChainlinkPriceFeed".

```
14 interface IChainlinkPriceFeed {
```



CVF-356 INFO

- **Category** Bad datatype
- **Source** PriceOracle.sol

Recommendation The type for this variable should be: mapping(IERC20 ⇒ IChainlinkPriceFeed)

```
26 mapping(address => address) public tokenToChainlinkFeed;
```

CVF-357 INFO

- **Category** Bad datatype
- **Source** PriceOracle.sol

Recommendation Events are usually named via nouns, such as "PriceFeed".

```
28 event PriceFeedUpdated(address token, address feed);
```

CVF-358 INFO

- **Category** Bad datatype
- **Source** PriceOracle.sol

Recommendation The type for the "token" parameter should be "IERC20".

```
28 event PriceFeedUpdated(address token, address feed);
```

```
31 error FeedNotFound(address token);
```

CVF-359 INFO

- **Category** Bad datatype
- **Source** PriceOracle.sol

Recommendation The type for the "feed" parameter should be "IChainlinkPriceFeed".

```
28 event PriceFeedUpdated(address token, address feed);
```

CVF-360 INFO

- **Category** Procedural
- **Source** PriceOracle.sol

Recommendation Both parameters should be indexed.

```
28 event PriceFeedUpdated(address token, address feed);
```

CVF-361 INFO

- **Category** Bad datatype
- **Source** PriceOracle.sol

Recommendation The type for the "token" argument should be "IERC20".

```
37 function setPriceFeed(address token, address feed) external onlyRole  
    ↪ (GOVERNANCE_ROLE) {
```

```
54 function getUsdValue(address token, uint256 amount) external view  
    ↪ override returns (uint256) {
```

```
59 function getUsdValueUnified(address token, uint256 amount) external  
    ↪ view override returns (uint256) {
```

CVF-362 INFO

- **Category** Bad datatype
- **Source** PriceOracle.sol

Recommendation The type for the "feed" argument should be "IChainlinkPriceFeed".

```
37 function setPriceFeed(address token, address feed) external onlyRole  
    ↪ (GOVERNANCE_ROLE) {
```

CVF-363 INFO

- **Category** Suboptimal
- **Source** PriceOracle.sol

Description These checks are redundant, as it is anyway possible to pass a dead token address.

Recommendation Remove these checks.

```
38 require(token != address(0), Errors.ZeroAddress());
```

```
44 require(token != address(0), Errors.ZeroAddress());
```

CVF-364 INFO

- **Category** Unclear behavior
- **Source** PriceOracle.sol

Description This event is emitted even if nothing actually changed.

```
40 emit PriceFeedUpdated(token, feed);
```

CVF-365 INFO

- **Category** Bad datatype
- **Source** PriceOracle.sol

Recommendation The argument type should be "IERC20".

```
43 function getPrice(address token) public view override returns (  
    ↪ uint256, uint256) {
```

CVF-366 INFO

- **Category** Bad naming
- **Source** PriceOracle.sol

Description The semantics of the returned values is unclear.

Recommendation Give descriptive names to the returned values and/or explain in a documentation comment.

```
43 function getPrice(address token) public view override returns (  
    ↪ uint256, uint256) {
```

CVF-367 INFO

- **Category** Bad datatype
- **Source** ReshufflingGateway.sol

Recommendation The type for this variable should be "IVault".

```
30 address public vault;
```

CVF-368 INFO

- **Category** Bad datatype
- **Source** ReshufflingGateway.sol

Recommendation The type for this variable should be "IERC20".

```
31 address public notion;
```

CVF-369 INFO

- **Category** Bad datatype
- **Source** ReshufflingGateway.sol

Recommendation The type for this variable should be "ISwapRouter".

```
32 address public swapRouter;
```



CVF-370 INFO

- **Category** Bad datatype
- **Source** ReshufflingGateway.sol

Recommendation The type for this argument should be "IVault".

```
57 address _vault,
```

CVF-371 INFO

- **Category** Bad datatype
- **Source** ReshufflingGateway.sol

Recommendation The type for this argument should be "IERC20".

```
58 address _notion,
```

```
71 function whitelistToken(address token) external onlyRole(  
    ↪ WHITELIST_MANAGER_ROLE) {
```

```
82 function blacklistToken(address token) external onlyRole(  
    ↪ WHITELIST_MANAGER_ROLE) {
```

CVF-372 INFO

- **Category** Bad datatype
- **Source** ReshufflingGateway.sol

Recommendation The type for this argument should be "ISwapRouter".

```
59 address _swapRouter,
```

CVF-373 INFO

- **Category** Bad datatype
- **Source** ReshufflingGateway.sol

Recommendation The argument type should be "IBridgeAdapter".

```
76 function whitelistBridgeAdapter(address bridgeAdapter) external  
    ↪ onlyRole(WHITELIST_MANAGER_ROLE) {
```

```
88 function blacklistBridgeAdapter(address bridgeAdapter) external  
    ↪ onlyRole(WHITELIST_MANAGER_ROLE) {
```

CVF-374 INFO

- **Category** Unclear behavior
- **Source** ReshufflingGateway.sol

Description These events are emitted even if nothing actually changed.

```
78 emit BridgeAdapterWhitelisted(bridgeAdapter);
```

```
84 emit TokenBlacklisted(token);
```

CVF-375 INFO

- **Category** Procedural
- **Source** ReshufflingGateway.sol

Recommendation This TODO is confusing, and should be removed.

```
81 // todo: blacklist
```

```
87 // todo: blacklist
```

CVF-376 INFO

- **Category** Bad naming
- **Source** ReshufflingGateway.sol

Description The function name is confusing, as the function doesn't add the token into the black list, but just removes it from the white list. Actually, there is no such thing as the black list.

Recommendation Rename the function.

```
82 function blacklistToken(address token) external onlyRole(  
    ↪ WHITELIST_MANAGER_ROLE) {  
    _whitelistedTokens.remove(token);
```

```
88 function blacklistBridgeAdapter(address bridgeAdapter) external  
    ↪ onlyRole(WHITELIST_MANAGER_ROLE) {  
    _whitelistedBridgeAdapters.remove(bridgeAdapter);
```

CVF-377 INFO

- **Category** Bad datatype
- **Source** ReshufflingGateway.sol

Recommendation The type for the "bridgeAdapter" argument should be "IBridgeAdapter".

```
93 function claimBridge(address bridgeAdapter, address token) external  
    ↪ nonReentrant {
```

CVF-378 INFO

- **Category** Bad datatype
- **Source** ReshufflingGateway.sol

Recommendation The type for the "token" argument should be "IERC20".

```
93 function claimBridge(address bridgeAdapter, address token) external  
    ↪ nonReentrant {
```

CVF-379 INFO

- **Category** Bad datatype
- **Source** ReshufflingGateway.sol

Recommendation The type for this argument should be "IBridgeAdapter[]".

```
122 address[] memory bridgeAdapters,
```

CVF-380 INFO

- **Category** Suboptimal
- **Source** ReshufflingGateway.sol

Recommendation It would be more efficient to pass a single array of structures with two fields, rather than two parallel arrays. This would also make the length check unnecessary.

```
122 address[] memory bridgeAdapters,  
    IBridgeAdapter.BridgeInstruction[] calldata instructions
```

```
164 address[] memory tokens,  
    uint256[] memory amounts
```

CVF-381 INFO

- **Category** Bad datatype
- **Source** ReshufflingGateway.sol

Recommendation The type for this argument should be "IContainer".

```
163 address container,
```

CVF-382 INFO

- **Category** Bad datatype
- **Source** ReshufflingGateway.sol

Recommendation The type for this argument should be "IERC20".

```
164 address[] memory tokens,
```

CVF-383 INFO

- **Category** Bad datatype
- **Source** SwapRouter.sol

Recommendation The key type for this mappings should be "ISwapAdapter".

```
17 mapping(address => bool) public whitelistedAdapters;
```

CVF-384 INFO

- **Category** Bad datatype
- **Source** SwapRouter.sol

Recommendation The argument type should be "ISwapAdapter".

```
35 function whitelistSwapAdapter(address adapter) external  
    ↪ onlyWhitelistManager {
```

```
42 function blacklistSwapAdapter(address adapter) external  
    ↪ onlyWhitelistManager {
```

CVF-385 INFO

- **Category** Bad datatype
- **Source** SwapRouter.sol

Recommendation These events are emitted even if nothing actually changed.

```
38 emit SwapAdapterWhitelisted(adapter);
```

```
44 emit SwapAdapterBlacklisted(adapter);
```

CVF-386 FIXED

- **Category** Procedural
- **Source** SwapRouter.sol

Recommendation This TODO is confusing and should be removed.

```
41 // todo: blacklist
```

CVF-387 INFO

- **Category** Bad datatype
- **Source** SwapRouter.sol

Description The name is confusing, as this function doesn't add the adapter into the black list, but just removes it from the white list. There is no such thing as the black list in the contract.

Recommendation Rename the function.

```
42 function blacklistSwapAdapter(address adapter) external  
    ↪ onlyWhitelistManager {
```

CVF-388 FIXED

- **Category** Procedural
- **Source** SwapRouter.sol

Recommendation This comment doesn't explain anything and should be removed.

```
68 return amountOutAfter - amountOutBefore; // cache
```

CVF - 389 INFO

- **Category** Suboptimal

- **Source** Vault.sol

Recommendation It would be more efficient to merge these mappings into a single mapping whose keys are deposit batch IDs and values are structs encapsulating the values of the original mappings.

```
48 mapping(uint256 => mapping(address => uint256)) public  
    ↪ pendingBatchDeposits;  
mapping(uint256 => uint256) public depositBatchTotalNotion;  
50 mapping(uint256 => uint256) public depositBatchTotalShares;  
mapping(uint256 => uint256) public batchNotionRemainder;
```

CVF - 390 INFO

- **Category** Documentation

- **Source** Vault.sol

Description The semantics of the second key in this mapping is unclear.

Recommendation Give a descriptive names to the key and/or explain in a documentation comment.

```
48 mapping(uint256 => mapping(address => uint256)) public  
    ↪ pendingBatchDeposits;
```

```
60 mapping(uint256 => mapping(address => uint256)) public  
    ↪ pendingBatchWithdrawals;
```

CVF - 391 INFO

- **Category** Documentation

- **Source** Vault.sol

Description The semantics of the key in this mapping is unclear.

Recommendation Give a descriptive name to the key and/or explain in a documentation comment.

```
54 mapping(address => ContainerReport) private _depositReports;
```

```
68 mapping(address => uint256) public containerWeights;
```

```
74 mapping(address => bool) private _hasClaimedReshufflingGateway;
```

CVF-392 INFO

- **Category** Suboptimal

- **Source** Vault.sol

Recommendation It would be more efficient to merge these mappings into a single mapping whose keys are withdrawal batch IDs and values are structs encapsulating the values of the original mappings.

```
60 mapping(uint256 => mapping(address => uint256)) public
    ↪ pendingBatchWithdrawals;
    mapping(uint256 => uint256) public withdrawBatchTotalShares;
    mapping(uint256 => uint256) public withdrawBatchTotalNotion;
```

CVF-393 INFO

- **Category** Bad datatype

- **Source** Vault.sol

Recommendation The key type for this mapping should be more specific.

```
68 mapping(address => uint256) public containerWeights;
```

```
74 mapping(address => bool) private _hasClaimedReshufflingGateway;
```

CVF-394 INFO

- **Category** Bad datatype

- **Source** Vault.sol

Recommendation The type for this argument should be "IERC20".

```
107 address _notion,
```

CVF-395 INFO

- **Category** Suboptimal

- **Source** Vault.sol

Description These checks are redundant, as it is anyway possible to pass dead addresses.

Recommendation Remove these checks.

```
115 require(roleAddresses.defaultAdmin != address(0), Errors.ZeroAddress
    ↪ ());
require(roleAddresses.containerManager != address(0), Errors.
    ↪ ZeroAddress());
require(roleAddresses.operator != address(0), Errors.ZeroAddress());
require(roleAddresses.configurator != address(0), Errors.ZeroAddress
    ↪ ());
require(roleAddresses.emergencyManager != address(0), Errors.
    ↪ ZeroAddress());

133 require(_notion != address(0), Errors.ZeroAddress());

231 require(container != address(0), Errors.ZeroAddress());

318 require(onBehalfOf != address(0), Errors.ZeroAddress());

371 require(onBehalfOf != address(0), Errors.ZeroAddress());
```

CVF-396 INFO

- **Category** Unclear behavior
- **Source** Vault.sol

Description These events are emitted even if nothing actually changed.

```
143 emit ReshufflingGatewayUpdated(previousGateway, _reshufflingGateway)
    ↪ ;
156 emit RepairingModeSet();
185 emit MaxDepositAmountUpdated(_maxDepositAmount);
191 emit MinDepositAmountUpdated(_minDepositAmount);
197 emit MaxDepositBatchSizeUpdated(_maxDepositBatchSize);
206 emit MinDepositBatchSizeUpdated(_minDepositBatchSize);
212 emit MinWithdrawBatchRatioUpdated(_minWithdrawBatchRatio);
```

CVF-397 INFO

- **Category** Suboptimal
- **Source** Vault.sol

Recommendation The error name doesn't match the actual problem. Consider using another error.

```
148 require(isReshuffling != _isReshuffling, Errors.IncorrectInput());
```

CVF-398 INFO

- **Category** Documentation
- **Source** Vault.sol

Description The semantics of the returned values is unclear.

Recommendation Give descriptive names to the returned values and/or explain in a documentation comment.

```
217 function getContainers() external view override returns (address[]
    ↪ memory, uint256[] memory) {
```

CVF-399 INFO

- **Category** Bad datatype
- **Source** Vault.sol

Recommendation The type for the first returned value should be more specific.

```
217 function getContainers() external view override returns (address[]  
    ↪ memory, uint256[] memory) {
```

CVF-400 INFO

- **Category** Suboptimal
- **Source** Vault.sol

Recommendation It would be more efficient to return a single array of structures with two fields, rather than two parallel arrays.

```
217 function getContainers() external view override returns (address[]  
    ↪ memory, uint256[] memory) {
```

CVF-401 INFO

- **Category** Bad datatype
- **Source** Vault.sol

Recommendation The argument type should be more specific.

```
230 function addContainer(address container) external nonReentrant  
    ↪ onlyRole(CONTAINER_MANAGER_ROLE) {
```

CVF-402 INFO

- **Category** Bad datatype
- **Source** Vault.sol

Recommendation The type for this argument should be more specific.

```
246 address[] calldata containers,
```

```
274 function isContainer(address container) external view override  
    ↪ returns (bool) {
```

```
278 function _isContainer(address container) internal view returns (bool  
    ↪ ) {
```

CVF-403 INFO

- **Category** Suboptimal
- **Source** Vault.sol

Recommendation It would be more efficient to use a single array of structures with two fields, rather than two parallel arrays. This would also make the length check unnecessary.

```
246 address[] calldata containers,  
    uint256[] calldata weights
```

CVF-404 INFO

- **Category** Flaw
- **Source** Vault.sol

Recommendation A better name for this function would be "redeem".

```
352 function withdraw(uint256 sharesPercent) external nonReentrant  
    ↪ notInReshufflingMode {
```

CVF-405 INFO

- **Category** Procedural
- **Source** Vault.sol

Recommendation This check should be performed earlier.

```
431 require(vars.containersNumber > 0, NoContainers());
```



ABDK

Consulting

About us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and co-authored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, JavaScript, and other languages.

Contact

Email

dmitry@abdkconsulting.com

Website

abdk.consulting

Twitter

twitter.com/ABDKconsulting

LinkedIn

linkedin.com/company/abdk-consulting